

Behavioral Anomaly Detection

Source Code Listing (generated for hackathon submission)

```
.
+-- README.md
+-- requirements.txt
+-- src/
|   +-- __init__.py
|   +-- common/
|   |   +-- __init__.py
|   |   +-- geo.py
|   +-- generator/
|   |   +-- __init__.py
|   |   +-- config.py
|   |   +-- entities.py
|   |   +-- sessions.py
|   |   +-- attacks.py
|   |   +-- generate_dataset.py
|   +-- features/
|   |   +-- __init__.py
|   |   +-- build_features.py
|   |   +-- drift.py
|   +-- models/
|   |   +-- __init__.py
|   |   +-- baseline.py
|   |   +-- classifier.py
|   |   +-- cold_start.py
|   |   +-- persistence.py
|   |   +-- sequence_model.py
|   +-- explain/
|   |   +-- __init__.py
|   |   +-- attribution.py
|   |   +-- narrative.py
|   +-- dashboard/
|   |   +-- __init__.py
|   |   +-- app.py
|   |   +-- feedback.py
|   +-- demo/
|   |   +-- __init__.py
|   |   +-- coldstart_drift.py
|   +-- pipeline.py
+-- tests/
|   +-- __init__.py
|   +-- conftest.py
|   +-- test_generator.py
|   +-- test_features.py
|   +-- test_models.py
|   +-- test_cold_start.py
|   +-- test_feedback.py
```

README.md

```
1 # Behavioral Anomaly Detection
2
3 AI/ML system for detecting and classifying anomalous access behavior (credential
4 misuse, lateral movement, brute force, impossible travel, device spoofing) from
5 entity access logs, with an explainable risk score for each alert.
6
7 Since real access-log/intrusion data is scarce and privacy-restricted, this
8 project generates its own synthetic dataset with injected attack patterns and
9 ground-truth labels, then trains detection and classification models on top of
10 it.
11
12 ## What's inside
13
14 - `src/generator/` - synthetic access-log generator: per-entity behavioral
15   profiles (users, service accounts, edge devices) plus six injected attack
16   patterns (including a low-and-slow exfiltration pattern that builds up
17   gradually in off-hours over days to weeks) plus an ambiguous "insider
18   drift" edge case - seven injected labels in total.
19 - `src/features/` - turns raw session rows into per-session features
20   (geo-velocity, resource novelty, device fingerprint mismatch, rolling
21   failure/breadth windows) with a drift-tolerant rolling baseline instead of a
22   fixed all-time one.
23 - `src/models/` - three complementary models:
24   - `baseline.py` - per-entity-type isolation forest plus a per-entity
25     statistical profile
26   - `classifier.py` - random forest over tabular features (multi-class
27     anomaly type, SHAP-friendly)
28   - `sequence_model.py` - small GRU over each entity's recent session window
29   - `cold_start.py` - blends a new entity's score toward its population
30     baseline until it has enough history of its own
31 - `src/explain/` - SHAP attribution composed into a one-sentence explanation
32   per alert, with the raw values kept alongside it.
33 - `src/pipeline.py` - runs the whole thing end to end and writes scored
34   sessions + metrics to `data/processed/`.
35 - `src/demo/coldstart_drift.py` - reproduces the cold-start and drift evidence
36   in the report as two charts.
37 - `src/dashboard/app.py` - Streamlit analyst view: ranked alert queue, alert
38   detail with explanation and entity history, confirm/dismiss triage, and a
39   system-behaviour tab.
40 - `tests/` - pytest coverage for the generator, features, models, cold-start
41   blending, and the feedback loop.
42
43 ## Setup
44
45 Requires Python 3.11+.
46
47 ```bash
48 python -m venv .venv
49 .venv/Scripts/activate
50 pip install -r requirements.txt
51 ```
52
53 ## Running it
54
55 Generate the synthetic dataset and run the full pipeline (this also generates
```

```

56 the data on first run if `data/raw/` is empty):
57
58 ```bash
59 python -m src.pipeline
60 ```
61
62 This writes `data/raw/access_log.csv`, `data/raw/labels.csv`,
63 `data/raw/entities.csv`, and `data/processed/scored_sessions.csv` +
64 `data/processed/metrics.json`.
65
66 To regenerate the dataset on its own (e.g. with a different seed):
67
68 ```bash
69 python -m src.generator.generate_dataset --seed 7
70 ```
71
72 Reproduce the cold-start and concept-drift charts used in the report (needs the
73 pipeline to have run once, since it reuses the trained models):
74
75 ```bash
76 python -m src.demo.coldstart_drift
77 ```
78
79 Launch the dashboard once `data/processed/` exists:
80
81 ```bash
82 streamlit run src/dashboard/app.py
83 ```
84
85 Run tests:
86
87 ```bash
88 pytest
89 ```
90
91 ## Approach
92
93 Full assumptions, metrics, and known limitations are in
94 [reports/report.md](reports/report.md).
95
96 Short version: three models score every session (an isolation forest baseline,
97 a random forest over tabular features, and a GRU over each entity's recent
98 session window), blended into one risk score. New entities get their score
99 pulled toward the population baseline until they build up history - without
100 that, a device's first session scores 0.57 purely for being unfamiliar. Two
101 z-scored features (session duration, login hour) use a trailing 30-day window
102 per entity rather than all-time history, so a permanent behavior shift stops
103 being flagged after it's been the norm for a while. A separate 7-day resource-
104 breadth window catches slow, gradual resource accumulation that a 24-hour
105 window would miss - the signature of the low-and-slow exfiltration pattern.
106
107 Each alert carries a generated one-line explanation ("Flagged as brute force
108 due to 47 failed logins for this entity within 10 minutes, combined with a 100%
109 authentication failure rate from this source IP"), with the SHAP values behind
110 it available in the dashboard. Analysts can confirm or dismiss alerts;
111 dismissals push that entity down the queue by a capped, documented offset.
112

```

113 The report is explicit about where this falls short. The one that matters most:
114 the risk score and the type classifier disagree for half the attack types -
115 device spoofing is typed correctly 99.4% of the time but reaches the top-1%
116 queue 1.8% of the time. Also: resource-footprint drift is never forgiven, and
117 precision degrades sharply past a top-2% alert budget.
118
119 The feedback loop's abuse cap is measured in percentile rank rather than raw
120 score, because a fixed score offset bounded almost nothing - it was wider than
121 the cutoff margin for 96% of queued entities. Capping queue movement instead
122 cut the share of entities an analyst can silently clear from 95.8% to 28.2%.
123 That is a reduction rather than a fix, and the report is explicit that what
124 remains is skewed toward low-footprint attackers.

requirements.txt

```
1  numpy>=1.26
2  pandas>=2.1,<3.0
3  faker>=22.0
4  scikit-learn>=1.4
5  torch>=2.2
6  shap>=0.44
7  streamlit>=1.31
8  pytest>=8.0
9  matplotlib>=3.8
```

src/common/geo.py

```
1 import numpy as np
2
3
4 def haversine_km(lat1, lon1, lat2, lon2):
5     r = 6371.0
6     lat1, lon1, lat2, lon2 = map(np.radians, [lat1, lon1, lat2, lon2])
7     dlat = lat2 - lat1
8     dlon = lon2 - lon1
9     a = np.sin(dlat / 2) ** 2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon / 2) ** 2
10    return 2 * r * np.arcsin(np.sqrt(a))
```

src/generator/config.py

```
1 from datetime import datetime, timedelta
2
3 SEED = 42
4
5 N_USERS = 250
6 N_SERVICE_ACCOUNTS = 40
7 N_EDGE_DEVICES = 80
8
9 SIM_DAYS = 45
10 SIM_START = datetime(2026, 5, 1)
11 SIM_END = SIM_START + timedelta(days=SIM_DAYS)
12
13 # fraction of total normal sessions each attack type contributes, roughly
14 # 0.5-3% of sessions per the brief. insider_drift is an ambiguous edge case,
15 # not a hard "anomaly" ground truth -- kept separate for FP-rate tuning.
16 ATTACK_RATES = {
17     "brute_force": 0.006,
18     "impossible_travel": 0.004,
19     "credential_stuffing": 0.005,
20     "lateral_movement": 0.004,
21     "device_spoofing": 0.003,
22     "insider_drift": 0.006,
23     "low_and_slow_exfil": 0.004,
24 }
25
26 ANOMALY_TYPES = list(ATTACK_RATES.keys())
27 ALL_LABELS = ["normal"] + ANOMALY_TYPES
28
29 AUTH_METHODS = ["password", "token", "certificate", "biometric"]
30
31 USER_RESOURCES = [
32     "hr/payroll", "finance/ledger", "shared-drive/marketing", "shared-drive/eng",
33     "crm/accounts", "crm/leads", "wiki/engineering", "wiki/hr", "vpn-gateway",
34     "email/inbox", "billing-portal", "reports/quarterly", "reports/sales",
35     "code-repo/frontend", "code-repo/backend", "admin/user-management",
36     "admin/access-control", "file-server/legal", "file-server/design",
37 ]
38
39 SERVICE_RESOURCES = [
40     "svc-auth:8443", "svc-billing:5432", "svc-orders:443", "svc-inventory:8080",
41     "svc-notifications:5672", "svc-search:9200", "svc-cache:6379",
42     "svc-logging:9000", "svc-metrics:9090", "svc-gateway:443",
```

```

43 ]
44
45 EDGE_RESOURCES = [
46     "telemetry-upload", "firmware-check", "sensor-read", "actuator-control",
47     "config-sync", "heartbeat", "diagnostics-report", "ota-update-check",
48 ]
49
50 RESOURCES_BY_TYPE = {
51     "user": USER_RESOURCES,
52     "service_account": SERVICE_RESOURCES,
53     "edge_device": EDGE_RESOURCES,
54 }
55
56 PRIVILEGED_ACTIONS = [
57     "list_dir", "read_file", "write_file", "delete_file", "run_query",
58     "export_data", "create_user", "modify_permissions", "restart_service",
59     "read_config", "write_config", "escalate_privilege",
60 ]
61
62 DEVICE_OS = [
63     "Ubuntu20.04", "Ubuntu22.04", "Windows10", "Windows11", "macOS14",
64     "EdgeOS-3.2", "EdgeOS-4.0", "RTOS-1.8",
65 ]

```

src/generator/entities.py

```

1  import random
2
3  import numpy as np
4  from faker import Faker
5
6  from src.generator import config
7
8
9  def _random_mac(rng):
10     return ":".join(f"{rng.integers(0, 256):02X}" for _ in range(6))
11
12
13  def _make_profile(entity_id, entity_type, fake, rng):
14     resources = config.RESOURCES_BY_TYPE[entity_type]
15     n_typical = {"user": 4, "service_account": 3, "edge_device": 2}[entity_type]
16     typical_resources = list(rng.choice(resources, size=min(n_typical, len(resources)),
17     replace=False))
18
19     city = fake.city()
20     country = fake.country()
21     lat, lon = float(fake.latitude()), float(fake.longitude())
22     home_ip = fake.ipv4_public()
23
24     if entity_type == "user":
25         hour_mean = rng.uniform(7.5, 10.5)
26         hour_std = rng.uniform(0.7, 1.8)
27         active_weekdays = list(range(5))
28         sessions_per_active_day = rng.uniform(2, 6)
29         duration_mean = rng.uniform(300, 2400)
30         auth_method = rng.choice(config.AUTH_METHODS, p=[0.35, 0.35, 0.1, 0.2])
31         privileged = rng.random() < 0.12

```

```

32     os_name = rng.choice(config.DEVICE_OS[:5])
33     mac = _random_mac(rng)
34     elif entity_type == "service_account":
35         hour_mean = 12.0
36         hour_std = 8.0
37         active_weekdays = list(range(7))
38         sessions_per_active_day = rng.uniform(15, 45)
39         duration_mean = rng.uniform(5, 120)
40         auth_method = "certificate" if rng.random() < 0.7 else "token"
41         privileged = True
42         os_name = "linux-container"
43         mac = _random_mac(rng)
44     else:
45         hour_mean = 12.0
46         hour_std = 7.0
47         active_weekdays = list(range(7))
48         sessions_per_active_day = rng.uniform(4, 12)
49         duration_mean = rng.uniform(2, 30)
50         auth_method = "certificate"
51         privileged = False
52         os_name = rng.choice(config.DEVICE_OS[:5])
53         mac = _random_mac(rng)
54
55     return {
56         "entity_id": entity_id,
57         "entity_type": entity_type,
58         "home_city": city,
59         "home_country": country,
60         "home_lat": lat,
61         "home_lon": lon,
62         "home_ip": home_ip,
63         "hour_mean": hour_mean,
64         "hour_std": hour_std,
65         "active_weekdays": active_weekdays,
66         "sessions_per_active_day": sessions_per_active_day,
67         "duration_mean": duration_mean,
68         "duration_std": duration_mean * 0.35,
69         "auth_method": auth_method,
70         "typical_resources": typical_resources,
71         "all_resources": resources,
72         "privileged": privileged,
73         "os_name": os_name,
74         "mac": mac,
75         "joined_day": 0,
76     }
77
78
79 def build_entities(seed=config.SEED):
80     Faker.seed(seed)
81     random.seed(seed)
82     rng = np.random.default_rng(seed)
83     fake = Faker()
84
85     profiles = {}
86     for i in range(config.N_USERS):
87         eid = f"user_{i:04d}"
88         profiles[eid] = _make_profile(eid, "user", fake, rng)

```

```

89     for i in range(config.N_SERVICE_ACCOUNTS):
90         eid = f"svc_{i:04d}"
91         profiles[eid] = _make_profile(eid, "service_account", fake, rng)
92     for i in range(config.N_EDGE_DEVICES):
93         eid = f"dev_{i:04d}"
94         profiles[eid] = _make_profile(eid, "edge_device", fake, rng)
95
96     # a handful of entities "join" partway through the simulation window to
97     # exercise the cold-start path downstream.
98     n_cold_start = max(1, min(len(profiles), round(len(profiles) * 0.07)))
99     cold_start_ids = rng.choice(list(profiles.keys()), size=n_cold_start, replace=False)
100    for eid in cold_start_ids:
101        profiles[eid]["joined_day"] = int(rng.integers(config.SIM_DAYS - 10, config.SIM_DAYS - 1))
102
103    return profiles

```

src/generator/sessions.py

```

1  import uuid
2  from datetime import timedelta
3
4  import numpy as np
5
6  from src.generator import config
7
8
9  def _sample_hour(rng, mean, std):
10     hour = rng.normal(mean, std) % 24
11     return hour
12
13
14  def _sample_command_sequence(rng, profile):
15     if not profile["privileged"]:
16         return []
17     length = rng.integers(2, 7)
18     return list(rng.choice(config.PRIVILEGED_ACTIONS, size=length, replace=True))
19
20
21  def _jitter_geo(rng, lat, lon, scale=0.15):
22     return lat + rng.normal(0, scale), lon + rng.normal(0, scale)
23
24
25  def generate_normal_sessions(profiles, seed=config.SEED):
26     rng = np.random.default_rng(seed + 1)
27     rows = []
28
29     for eid, profile in profiles.items():
30         day = profile["joined_day"]
31         while day < config.SIM_DAYS:
32             weekday = (config.SIM_START + timedelta(days=day)).weekday()
33             if weekday not in profile["active_weekdays"]:
34                 day += 1
35                 continue
36             if rng.random() > 0.9 and profile["entity_type"] == "user":
37                 day += 1
38                 continue
39

```

```

40     n_sessions = max(0, int(rng.poisson(profile["sessions_per_active_day"])))
41     for _ in range(n_sessions):
42         hour = _sample_hour(rng, profile["hour_mean"], profile["hour_std"])
43         ts = (
44             config.SIM_START
45             + timedelta(days=day, hours=hour)
46             + timedelta(seconds=int(rng.integers(0, 60)))
47         )
48
49         traveling = profile["entity_type"] == "user" and rng.random() < 0.02
50         if traveling:
51             lat, lon = _jitter_geo(rng, profile["home_lat"], profile["home_lon"],
52 scale=8.0)
53             source_ip = f"{rng.integers(1, 223)}.{rng.integers(0, 256)}.{rng.integers(0,
54 256)}.{rng.integers(1, 255)}"
55         else:
56             lat, lon = _jitter_geo(rng, profile["home_lat"], profile["home_lon"])
57             source_ip = profile["home_ip"]
58
59         if rng.random() < 0.85:
60             resource = rng.choice(profile["typical_resources"])
61         else:
62             resource = rng.choice(profile["all_resources"])
63
64         duration = max(1.0, rng.normal(profile["duration_mean"], profile["duration_std"]))
65         auth_success = rng.random() > 0.02
66         auth_method = profile["auth_method"] if rng.random() > 0.05 else
67 rng.choice(config.AUTH_METHODS)
68
69         rows.append({
70             "session_id": str(uuid.uuid4()),
71             "entity_id": eid,
72             "entity_type": profile["entity_type"],
73             "timestamp": ts,
74             "source_ip": source_ip,
75             "geo_lat": lat,
76             "geo_lon": lon,
77             "geo_city": profile["home_city"] if not traveling else "unknown-transit",
78             "resource_accessed": resource,
79             "auth_method": auth_method,
80             "auth_success": auth_success,
81             "session_duration": duration,
82             "command_sequence": "|".join(_sample_command_sequence(rng, profile)),
83             "device_os": profile["os_name"],
84             "device_mac": profile["mac"],
85             "label": "normal",
86         })
87         day += 1
88
89     return rows

```

src/generator/attacks.py

```

1 import uuid
2 from datetime import timedelta
3
4 import numpy as np

```

```

5 import pandas as pd
6
7 from src.common.geo import haversine_km
8 from src.generator import config
9
10
11 def _random_ip(rng):
12     return f"{rng.integers(1, 223)}.{rng.integers(0, 256)}.{rng.integers(0, 256)}.{rng.integers(1,
13 255)}"
14
15
16 def _random_far_geo(rng, lat, lon, min_km=3000):
17     while True:
18         cand_lat = rng.uniform(-60, 70)
19         cand_lon = rng.uniform(-180, 180)
20         if haversine_km(lat, lon, cand_lat, cand_lon) >= min_km:
21             return cand_lat, cand_lon
22
23
24 def _row(entity_id, entity_type, ts, ip, lat, lon, city, resource, auth_method,
25         success, duration, commands, os_name, mac, label):
26     return {
27         "session_id": str(uuid.uuid4()),
28         "entity_id": entity_id,
29         "entity_type": entity_type,
30         "timestamp": ts,
31         "source_ip": ip,
32         "geo_lat": lat,
33         "geo_lon": lon,
34         "geo_city": city,
35         "resource_accessed": resource,
36         "auth_method": auth_method,
37         "auth_success": success,
38         "session_duration": duration,
39         "command_sequence": "|".join(commands),
40         "device_os": os_name,
41         "device_mac": mac,
42         "label": label,
43     }
44
45
46 def inject_brute_force(profiles, rng, n_incidents):
47     rows = []
48     entity_ids = list(profiles.keys())
49     for _ in range(n_incidents):
50         target = profiles[rng.choice(entity_ids)]
51         attacker_ip = _random_ip(rng)
52         day = int(rng.integers(0, config.SIM_DAYS))
53         start = config.SIM_START + timedelta(days=day, hours=float(rng.uniform(0, 24)))
54         n_attempts = int(rng.integers(12, 60))
55         gap_seconds = rng.uniform(1, 8, size=n_attempts)
56         t = start
57         for i in range(n_attempts):
58             t = t + timedelta(seconds=float(gap_seconds[i]))
59             success = i == n_attempts - 1 and rng.random() < 0.3
60             rows.append(_row(
61                 target["entity_id"], target["entity_type"], t, attacker_ip,

```

```

62         target["home_lat"], target["home_lon"], "unknown-transit",
63         rng.choice(target["all_resources"]), "password", success,
64         max(1.0, rng.normal(5, 2)), [], target["os_name"], _random_mac_attacker(rng),
65         "brute_force",
66     ))
67     return rows
68
69
70 def _random_mac_attacker(rng):
71     return ":".join(f"{rng.integers(0, 256):02X}" for _ in range(6))
72
73
74 def inject_impossible_travel(profiles, normal_rows_df, rng, n_incidents):
75     rows = []
76     candidates = normal_rows_df[normal_rows_df["entity_type"] == "user"]
77     if candidates.empty:
78         return rows
79     for _ in range(n_incidents):
80         base = candidates.sample(1, random_state=int(rng.integers(0, 1_000_000))).iloc[0]
81         profile = profiles[base["entity_id"]]
82         gap_hours = rng.uniform(0.5, 3.0)
83         ts = base["timestamp"] + timedelta(hours=float(gap_hours))
84         if ts >= config.SIM_END:
85             continue
86         far_lat, far_lon = _random_far_geo(rng, base["geo_lat"], base["geo_lon"])
87         rows.append(_row(
88             base["entity_id"], profile["entity_type"], ts, _random_ip(rng),
89             far_lat, far_lon, "unknown-transit",
90             rng.choice(profile["typical_resources"]), profile["auth_method"], True,
91             max(1.0, rng.normal(profile["duration_mean"], profile["duration_std"])),
92             [], profile["os_name"], profile["mac"], "impossible_travel",
93         ))
94     return rows
95
96
97 def inject_credential_stuffing(profiles, rng, n_incidents):
98     rows = []
99     user_ids = [eid for eid, p in profiles.items() if p["entity_type"] == "user"]
100    for _ in range(n_incidents):
101        n_attacker_ips = int(rng.integers(1, 4))
102        attacker_ips = [_random_ip(rng) for _ in range(n_attacker_ips)]
103        n_targets = int(rng.integers(20, 70))
104        targets = rng.choice(user_ids, size=min(n_targets, len(user_ids)), replace=False)
105        day = int(rng.integers(0, config.SIM_DAYS))
106        start = config.SIM_START + timedelta(days=day, hours=float(rng.uniform(0, 24)))
107        t = start
108        for eid in targets:
109            profile = profiles[eid]
110            t = t + timedelta(seconds=float(rng.uniform(0.5, 6)))
111            success = rng.random() < 0.03
112            rows.append(_row(
113                eid, "user", t, rng.choice(attacker_ips),
114                profile["home_lat"], profile["home_lon"], "unknown-transit",
115                "vpn-gateway", "password", success,
116                max(1.0, rng.normal(4, 1.5)), [], profile["os_name"], _random_mac_attacker(rng),
117                "credential_stuffing",
118            ))

```

```

119     return rows
120
121
122 def inject_lateral_movement(profiles, rng, n_incidents):
123     rows = []
124     all_resources = sorted(set(sum(config.RESOURCE_BY_TYPE.values(), [])))
125     candidates = [eid for eid, p in profiles.items() if p["entity_type"] in ("user",
126 "service_account")]
127     for _ in range(n_incidents):
128         eid = rng.choice(candidates)
129         profile = profiles[eid]
130         day = int(rng.integers(profile["joined_day"], config.SIM_DAYS - 1))
131         start = config.SIM_START + timedelta(days=day, hours=float(rng.uniform(0, 24)))
132         unseen = [r for r in all_resources if r not in profile["typical_resources"]]
133         n_hops = int(rng.integers(6, 16))
134         targets = rng.choice(unseen, size=min(n_hops, len(unseen)), replace=False)
135         t = start
136         for resource in targets:
137             t = t + timedelta(minutes=float(rng.uniform(1, 10)))
138             commands = list(rng.choice(config.PRIVILEGED_ACTIONS, size=int(rng.integers(2, 5)),
139 replace=True))
140             rows.append(_row(
141                 eid, profile["entity_type"], t, profile["home_ip"],
142                 profile["home_lat"], profile["home_lon"], profile["home_city"],
143                 resource, profile["auth_method"], True,
144                 max(1.0, rng.normal(profile["duration_mean"] * 1.5, profile["duration_std"])),
145                 commands, profile["os_name"], profile["mac"], "lateral_movement",
146             ))
147     return rows
148
149
150 def inject_device_spoofing(profiles, rng, n_incidents):
151     rows = []
152     candidates = [eid for eid, p in profiles.items() if p["entity_type"] in ("edge_device",
153 "user")]
154     for _ in range(n_incidents):
155         eid = rng.choice(candidates)
156         profile = profiles[eid]
157         day = int(rng.integers(profile["joined_day"], config.SIM_DAYS - 1))
158         ts = config.SIM_START + timedelta(days=day, hours=float(rng.uniform(0, 24)))
159         fake_os = rng.choice([o for o in config.DEVICE_OS if o != profile["os_name"]])
160         rows.append(_row(
161             eid, profile["entity_type"], ts, _random_ip(rng),
162             profile["home_lat"], profile["home_lon"], profile["home_city"],
163             rng.choice(profile["typical_resources"]), profile["auth_method"], True,
164             max(1.0, rng.normal(profile["duration_mean"], profile["duration_std"])),
165             [], fake_os, _random_mac_attacker(rng), "device_spoofing",
166         ))
167     return rows
168
169
170 def inject_insider_drift(profiles, rng, n_incidents):
171     rows = []
172     candidates = [eid for eid, p in profiles.items() if p["entity_type"] == "user"]
173     for _ in range(n_incidents):
174         eid = rng.choice(candidates)
175         profile = profiles[eid]

```

```

176     unseen = [r for r in profile["all_resources"] if r not in profile["typical_resources"]]
177     if not unseen:
178         continue
179     start_day = int(rng.integers(profile["joined_day"], max(profile["joined_day"] + 1,
180 config.SIM_DAYS - 20)))
181     n_steps = int(rng.integers(8, 20))
182     for step in range(n_steps):
183         day = min(config.SIM_DAYS - 1, start_day + step * int(rng.integers(1, 3)))
184         hour = _sample_hour_like(profile, rng)
185         ts = config.SIM_START + timedelta(days=day, hours=hour)
186         resource = unseen[min(step // 2, len(unseen) - 1)]
187         rows.append(_row(
188             eid, "user", ts, profile["home_ip"],
189             profile["home_lat"], profile["home_lon"], profile["home_city"],
190             resource, profile["auth_method"], True,
191             max(1.0, rng.normal(profile["duration_mean"], profile["duration_std"])),
192             [], profile["os_name"], profile["mac"], "insider_drift",
193         ))
194     return rows
195
196
197 def _sample_hour_like(profile, rng):
198     return float(rng.normal(profile["hour_mean"], profile["hour_std"]) % 24)
199
200
201 def inject_low_and_slow_exfil(profiles, rng, n_incidents):
202     """Small resource-access bursts concentrated in off-hours, accumulating
203 gradually over days to weeks rather than in one spike. Distinct from
204 lateral_movement (many resources hopped in a single short burst) and from
205 insider_drift (which samples hours from the entity's own normal
206 distribution, not an off-hours window - drift looks routine, this
207 doesn't)."""
208     rows = []
209     candidates = [eid for eid, p in profiles.items() if p["entity_type"] in ("user",
210 "service_account")]
211     for _ in range(n_incidents):
212         eid = rng.choice(candidates)
213         profile = profiles[eid]
214         unseen = [r for r in profile["all_resources"] if r not in profile["typical_resources"]]
215         if not unseen:
216             continue
217         span_days = int(rng.integers(10, 25))
218         start_day = int(rng.integers(
219             profile["joined_day"], max(profile["joined_day"] + 1, config.SIM_DAYS - span_days)
220         ))
221         off_hour_center = (profile["hour_mean"] + 12) % 24
222
223         touched = []
224         for day_offset in range(span_days):
225             day = start_day + day_offset
226             if day >= config.SIM_DAYS:
227                 break
228             if rng.random() < 0.6:
229                 continue # gradual: most days see nothing at all
230             n_sessions_today = int(rng.integers(1, 3))
231             for _ in range(n_sessions_today):
232                 hour = float(rng.normal(off_hour_center, 1.5) % 24)

```

```

233         ts = config.SIM_START + timedelta(days=day, hours=hour)
234         if len(touched) < len(unseen) and (not touched or rng.random() < 0.5):
235             resource = unseen[len(touched)]
236             touched.append(resource)
237         else:
238             resource = rng.choice(touched) if touched else unseen[0]
239         commands = ["export_data"] if rng.random() < 0.3 else []
240         rows.append(_row(
241             eid, profile["entity_type"], ts, profile["home_ip"],
242             profile["home_lat"], profile["home_lon"], profile["home_city"],
243             resource, profile["auth_method"], True,
244             max(1.0, rng.normal(profile["duration_mean"] * 1.3, profile["duration_std"])),
245             commands, profile["os_name"], profile["mac"], "low_and_slow_exfil",
246         ))
247     return rows
248
249
250 def inject_all(profiles, normal_rows, rng):
251     normal_df = pd.DataFrame(normal_rows)
252     total_normal = len(normal_rows)
253
254     rows = []
255     rows += inject_brute_force(
256         profiles, rng, max(1, int(total_normal * config.ATTACK_RATES["brute_force"] / 30)))
257     rows += inject_impossible_travel(
258         profiles, normal_df, rng, max(1, int(total_normal *
259 config.ATTACK_RATES["impossible_travel"])))
260     rows += inject_credential_stuffing(
261         profiles, rng, max(1, int(total_normal * config.ATTACK_RATES["credential_stuffing"] /
262 40)))
263     rows += inject_lateral_movement(
264         profiles, rng, max(1, int(total_normal * config.ATTACK_RATES["lateral_movement"] / 10)))
265     rows += inject_device_spoofing(
266         profiles, rng, max(1, int(total_normal * config.ATTACK_RATES["device_spoofing"])))
267     rows += inject_insider_drift(
268         profiles, rng, max(1, int(total_normal * config.ATTACK_RATES["insider_drift"] / 12)))
269     rows += inject_low_and_slow_exfil(
270         profiles, rng, max(1, int(total_normal * config.ATTACK_RATES["low_and_slow_exfil"] / 10)))
271     return rows

```

src/generator/generate_dataset.py

```

1  import argparse
2  from pathlib import Path
3
4  import numpy as np
5  import pandas as pd
6
7  from src.generator import config
8  from src.generator.attacks import inject_all
9  from src.generator.entities import build_entities
10 from src.generator.sessions import generate_normal_sessions
11
12 OUT_DIR = Path(__file__).resolve().parents[2] / "data" / "raw"
13
14 COLUMN_ORDER = [
15     "session_id", "entity_id", "entity_type", "timestamp", "source_ip",

```

```

16     "geo_lat", "geo_lon", "geo_city", "resource_accessed", "auth_method",
17     "auth_success", "session_duration", "command_sequence", "device_os",
18     "device_mac",
19 ]
20
21
22 def build_dataset(seed=config.SEED):
23     profiles = build_entities(seed)
24     normal_rows = generate_normal_sessions(profiles, seed)
25     attack_rows = inject_all(profiles, normal_rows, np.random.default_rng(seed + 2))
26
27     all_rows = normal_rows + attack_rows
28     df = pd.DataFrame(all_rows).sort_values("timestamp").reset_index(drop=True)
29
30     entities_df = pd.DataFrame([
31         {
32             "entity_id": eid,
33             "entity_type": p["entity_type"],
34             "home_city": p["home_city"],
35             "home_country": p["home_country"],
36             "home_ip": p["home_ip"],
37             "joined_day": p["joined_day"],
38             "typical_resources": "|".join(p["typical_resources"]),
39         }
40         for eid, p in profiles.items()
41     ])
42
43     return df, entities_df
44
45
46 def main():
47     parser = argparse.ArgumentParser()
48     parser.add_argument("--seed", type=int, default=config.SEED)
49     parser.add_argument("--out", type=str, default=str(OUT_DIR))
50     args = parser.parse_args()
51
52     out_dir = Path(args.out)
53     out_dir.mkdir(parents=True, exist_ok=True)
54
55     df, entities_df = build_dataset(args.seed)
56
57     labels = df[["session_id", "label"]].copy()
58     access_log = df[COLUMN_ORDER].copy()
59
60     access_log.to_csv(out_dir / "access_log.csv", index=False)
61     labels.to_csv(out_dir / "labels.csv", index=False)
62     entities_df.to_csv(out_dir / "entities.csv", index=False)
63
64     n_anomaly = (labels["label"] != "normal").sum()
65     print(f"sessions: {len(access_log)}")
66     print(f"anomalous sessions: {n_anomaly} ({n_anomaly / len(access_log):.3%})")
67     print(labels["label"].value_counts())
68
69
70 if __name__ == "__main__":
71     main()

```

src/features/build_features.py

```
1 import numpy as np
2 import pandas as pd
3
4 from src.common.geo import haversine_km
5 from src.features.drift import rolling_zscore
6
7 COLD_START_THRESHOLD = 5
8 PRIVILEGED_MARKERS = {"escalate_privilege", "delete_file", "modify_permissions", "create_user"}
9
10
11 def build_features(access_log, cold_start_threshold=COLD_START_THRESHOLD):
12     df = access_log.copy()
13     df["timestamp"] = pd.to_datetime(df["timestamp"])
14     df = df.sort_values(["entity_id", "timestamp"]).reset_index(drop=True)
15
16     entity_home = (
17         df.sort_values(["entity_id", "timestamp"])
18         .groupby("entity_id")[["geo_lat", "geo_lon"]]
19         .transform("first")
20     )
21     df["home_lat"] = entity_home["geo_lat"]
22     df["home_lon"] = entity_home["geo_lon"]
23
24     df["hour_of_day"] = df["timestamp"].dt.hour + df["timestamp"].dt.minute / 60
25     df["day_of_week"] = df["timestamp"].dt.dayofweek
26     df["is_weekend"] = (df["day_of_week"] >= 5).astype(int)
27
28     grp = df.groupby("entity_id", sort=False)
29
30     prev_ts = grp["timestamp"].shift(1)
31     df["hours_since_last_session"] = (df["timestamp"] - prev_ts).dt.total_seconds() / 3600
32     df["hours_since_last_session"] = df["hours_since_last_session"].fillna(9999.0)
33
34     prev_lat = grp["geo_lat"].shift(1)
35     prev_lon = grp["geo_lon"].shift(1)
36     dist_from_prev = haversine_km(prev_lat, prev_lon, df["geo_lat"], df["geo_lon"])
37     df["geo_distance_from_prev_km"] = dist_from_prev.fillna(0.0)
38
39     safe_hours = df["hours_since_last_session"].clip(lower=1 / 60)
40     df["geo_velocity_kmh"] = (df["geo_distance_from_prev_km"] / safe_hours).where(
41         df["hours_since_last_session"] < 9999.0, 0.0
42     )
43
44     df["geo_distance_from_home_km"] = haversine_km(
45         df["home_lat"], df["home_lon"], df["geo_lat"], df["geo_lon"]
46     )
47
48     df["entity_history_length"] = grp.cumcount()
49     df["is_cold_start"] = (df["entity_history_length"] < cold_start_threshold).astype(int)
50
51     is_first_resource = ~df.duplicated(subset=["entity_id", "resource_accessed"], keep="first")
52     df["is_new_resource"] = is_first_resource.astype(int)
53     df["is_new_resource"] = df["is_new_resource"].where(df["entity_history_length"] > 0, 0)
54
55     fingerprint = df["device_os"] + "::" + df["device_mac"]
```

```

56 baseline_fp = fingerprint.groupby(df["entity_id"], sort=False).transform("first")
57 df["is_device_mismatch"] = (fingerprint != baseline_fp).astype(int)
58
59 df["auth_failed"] = (~df["auth_success"].astype(bool)).astype(int)
60
61 df["session_duration_z"] = rolling_zscore(df, "session_duration")
62 df["hour_of_day_z"] = rolling_zscore(df, "hour_of_day")
63
64 df["command_sequence_length"] = df["command_sequence"].fillna("").apply(
65     lambda s: 0 if s == "" else len(s.split("|"))
66 )
67 df["has_privileged_command"] = df["command_sequence"].fillna("").apply(
68     lambda s: int(any(tok in PRIVILEGED_MARKERS for tok in s.split("|")))
69 )
70
71 # rolling().apply() needs numeric input, so id/resource columns are
72 # factorized to codes before the nunique-in-window computation.
73 df["_entity_code"] = pd.factorize(df["entity_id"])[0].astype(float)
74 df["_resource_code"] = pd.factorize(df["resource_accessed"])[0].astype(float)
75
76 df = df.sort_values(["source_ip", "timestamp"]).reset_index(drop=True)
77 ip_grp = df.groupby("source_ip", sort=False)
78 df["ip_session_count_10min"] = (
79     ip_grp.rolling("10min", on="timestamp")["session_id"].count().reset_index(drop=True) - 1
80 )
81 df["ip_distinct_entities_10min"] = (
82     ip_grp.rolling("10min", on="timestamp")["_entity_code"]
83     .apply(lambda s: len(set(s)), raw=True)
84     .reset_index(drop=True)
85     - 1
86 ).clip(lower=0)
87 df["ip_failure_rate_10min"] = (
88     ip_grp.rolling("10min", on="timestamp")["auth_failed"].mean().reset_index(drop=True)
89 )
90
91 df = df.sort_values(["entity_id", "timestamp"]).reset_index(drop=True)
92 grp = df.groupby("entity_id", sort=False)
93 df["entity_resource_breadth_24h"] = (
94     grp.rolling("24h", on="timestamp")["_resource_code"]
95     .apply(lambda s: len(set(s)), raw=True)
96     .reset_index(drop=True)
97 )
98 # a longer trailing window than the 24h one above: a burst of lateral
99 # movement spikes the 24h count, while a slow accumulation of new
100 # resources over days/weeks only shows up once the window is wide enough
101 # to hold the whole build-up.
102 df["entity_resource_breadth_7d"] = (
103     grp.rolling("7D", on="timestamp")["_resource_code"]
104     .apply(lambda s: len(set(s)), raw=True)
105     .reset_index(drop=True)
106 )
107 df["entity_failed_count_10min"] = (
108     grp.rolling("10min", on="timestamp")["auth_failed"].sum().reset_index(drop=True)
109 )
110 df = df.drop(columns=["_entity_code", "_resource_code"])
111
112 df = df.sort_values(["entity_id", "timestamp"]).reset_index(drop=True)

```

```

113     return df
114
115
116 FEATURE_COLUMNS = [
117     "hour_of_day", "day_of_week", "is_weekend", "hours_since_last_session",
118     "geo_distance_from_prev_km", "geo_velocity_kmh", "geo_distance_from_home_km",
119     "entity_history_length", "is_cold_start", "is_new_resource", "is_device_mismatch",
120     "auth_failed", "session_duration_z", "hour_of_day_z", "command_sequence_length",
121     "has_privileged_command", "ip_session_count_10min", "ip_distinct_entities_10min",
122     "ip_failure_rate_10min", "entity_resource_breadth_24h", "entity_resource_breadth_7d",
123     "entity_failed_count_10min",
124 ]

```

src/features/drift.py

```

1  import numpy as np
2
3  DRIFT_WINDOW = "30D"
4  MIN_PERIODS = 3
5
6
7  def rolling_zscore(df, value_col, window=DRIFT_WINDOW, min_periods=MIN_PERIODS):
8      """Trailing-window z-score per entity, excluding the current row, so a
9      baseline that shifted a month ago stops being penalized today instead of
10     staying anchored to all-time history."""
11     tmp = df[["entity_id", "timestamp", value_col]].copy()
12     tmp["_sq"] = tmp[value_col] ** 2
13
14     # groupby.rolling returns a (group, timestamp) multiindex; the group
15     # order matches tmp's row order since tmp is pre-sorted by entity/time,
16     # so a full positional reset lines these back up with tmp by row.
17     g = tmp.groupby("entity_id", sort=False)
18     roll_count = g.rolling(window, on="timestamp")[value_col].count().reset_index(drop=True)
19     roll_sum = g.rolling(window, on="timestamp")[value_col].sum().reset_index(drop=True)
20     roll_sumsq = g.rolling(window, on="timestamp")["_sq"].sum().reset_index(drop=True)
21
22     value = tmp[value_col].reset_index(drop=True)
23     n_excl = (roll_count - 1).clip(lower=0)
24     safe_n = n_excl.replace(0, np.nan)
25     mean_excl = (roll_sum - value) / safe_n
26     var_excl = (roll_sumsq - value ** 2) / safe_n - mean_excl ** 2
27     std_excl = np.sqrt(var_excl.clip(lower=0))
28
29     z = (value - mean_excl) / std_excl.replace(0, np.nan)
30     z = z.fillna(0.0)
31     z[n_excl < min_periods] = 0.0
32     return z.reset_index(drop=True)

```

src/models/baseline.py

```

1  import numpy as np
2  import pandas as pd
3  from sklearn.ensemble import IsolationForest
4  from sklearn.preprocessing import StandardScaler
5
6  from src.features.build_features import FEATURE_COLUMNS
7

```

```

8 COLD_START_THRESHOLD = 5
9
10
11 class BaselineProfiler:
12     """Population-level isolation forest per entity_type, plus a per-entity
13     statistical profile used for cold-start fallback and dashboard display."""
14
15     def __init__(self, contamination=0.02, random_state=42):
16         self.contamination = contamination
17         self.random_state = random_state
18         self.scalers = {}
19         self.forests = {}
20         self.entity_profiles = None
21         self.type_profiles = None
22
23     def fit(self, features_df):
24         for entity_type, group in features_df.groupby("entity_type"):
25             x = group[FEATURE_COLUMNS].values
26             scaler = StandardScaler().fit(x)
27             forest = IsolationForest(
28                 n_estimators=200,
29                 contamination=self.contamination,
30                 random_state=self.random_state,
31             ).fit(scaler.transform(x))
32             self.scalers[entity_type] = scaler
33             self.forests[entity_type] = forest
34
35             self.entity_profiles = self._build_entity_profiles(features_df)
36             self.type_profiles = self._build_type_profiles(features_df)
37         return self
38
39     def _build_entity_profiles(self, features_df):
40         agg = features_df.groupby("entity_id")[FEATURE_COLUMNS].agg(["mean", "std", "count"])
41         agg.columns = ["_".join(c) for c in agg.columns]
42         return agg
43
44     def _build_type_profiles(self, features_df):
45         agg = features_df.groupby("entity_type")[FEATURE_COLUMNS].agg(["mean", "std"])
46         agg.columns = ["_".join(c) for c in agg.columns]
47         return agg
48
49     def score(self, features_df):
50         scores = np.zeros(len(features_df))
51         for entity_type, group in features_df.groupby("entity_type"):
52             if entity_type not in self.forests:
53                 continue
54             x = self.scalers[entity_type].transform(group[FEATURE_COLUMNS].values)
55             raw = self.forests[entity_type].decision_function(x)
56             scores[group.index] = -raw
57
58         lo, hi = scores.min(), scores.max()
59         if hi - lo < 1e-9:
60             return np.zeros_like(scores)
61         return (scores - lo) / (hi - lo)
62
63     def entity_profile_summary(self, entity_id, entity_type):
64         if self.entity_profiles is not None and entity_id in self.entity_profiles.index:

```

```

65         row = self.entity_profiles.loc[entity_id]
66         if row.get("hour_of_day_count", 0) >= COLD_START_THRESHOLD:
67             return row, "entity"
68         if self.type_profiles is not None and entity_type in self.type_profiles.index:
69             return self.type_profiles.loc[entity_type], "population"
70         return None, "none"

```

src/models/classifier.py

```

1  from sklearn.ensemble import RandomForestClassifier
2
3  from src.features.build_features import FEATURE_COLUMNS
4  from src.generator.config import ALL_LABELS
5
6
7  class TabularClassifier:
8      """Random forest over the tabular session features. Slower-moving and
9      fully SHAP-explainable, used alongside the GRU for attribution and as a
10      sanity check on the sequence model's calls."""
11
12     def __init__(self, random_state=42):
13         self.model = RandomForestClassifier(
14             n_estimators=300,
15             max_depth=20,
16             class_weight="balanced",
17             random_state=random_state,
18             n_jobs=-1,
19         )
20         self.classes_ = None
21
22     def fit(self, features_df):
23         x = features_df[FEATURE_COLUMNS].values
24         y = features_df["label"].values
25         self.model.fit(x, y)
26         self.classes_ = list(self.model.classes_)
27         return self
28
29     def predict_proba(self, features_df):
30         x = features_df[FEATURE_COLUMNS].values
31         return self.model.predict_proba(x)
32
33     def predict(self, features_df):
34         x = features_df[FEATURE_COLUMNS].values
35         return self.model.predict(x)

```

src/models/cold_start.py

```

1  import numpy as np
2
3  from src.features.build_features import COLD_START_THRESHOLD
4
5
6  def blend_cold_start_scores(features_df, risk_scores, type_prior_scores,
7  threshold=COLD_START_THRESHOLD):
8      """New entities have too little history for their own profile or the
9      sequence model's context window to mean much, so their raw score is
10      unreliable in both directions. Blend it toward the entity_type's average

```

```

11     risk instead of trusting it outright, weighted by how much history exists."""
12     history = features_df["entity_history_length"].values
13     weight = np.clip(history / threshold, 0, 1)
14     prior = features_df["entity_type"].map(type_prior_scores).fillna(np.mean(risk_scores)).values
15     return weight * risk_scores + (1 - weight) * prior
16
17
18 def cold_start_note(history_length, threshold=COLD_START_THRESHOLD):
19     if history_length >= threshold:
20         return None
21     return (
22         f"Treat with caution: only {int(history_length)} prior sessions on record, "
23         "so the score is still partly the population baseline rather than this entity's own
    history."
    )

```

src/models/persistence.py

```

1  from pathlib import Path
2
3  import joblib
4
5  ROOT = Path(__file__).resolve().parents[2]
6  MODEL_DIR = ROOT / "models"
7  BUNDLE_NAME = "trained_bundle.joblib"
8
9
10 def save_bundle(bundle, model_dir=MODEL_DIR):
11     model_dir = Path(model_dir)
12     model_dir.mkdir(parents=True, exist_ok=True)
13     path = model_dir / BUNDLE_NAME
14     joblib.dump(bundle, path)
15     return path
16
17
18 def load_bundle(model_dir=MODEL_DIR):
19     path = Path(model_dir) / BUNDLE_NAME
20     if not path.exists():
21         raise FileNotFoundError(
22             f"no trained model bundle at {path} - run `python -m src.pipeline` first"
23         )
24     return joblib.load(path)

```

src/models/sequence_model.py

```

1  import numpy as np
2  import torch
3  import torch.nn as nn
4  from sklearn.preprocessing import StandardScaler
5
6  from src.features.build_features import FEATURE_COLUMNS
7  from src.generator.config import ALL_LABELS
8
9  WINDOW = 10
10
11
12 def build_sequences(features_df, window=WINDOW):

```

```

13 df = features_df.sort_values(["entity_id", "timestamp"]).reset_index(drop=True)
14 label_to_idx = {label: i for i, label in enumerate(ALL_LABELS)}
15 n_features = len(FEATURE_COLUMNS)
16
17 sequences = []
18 targets = []
19 session_ids = []
20
21 # Labels are training-only; at inference they are hidden, so the returned
22 # targets are placeholders in that case.
23 has_labels = "label" in df.columns
24
25 for entity_id, group in df.groupby("entity_id", sort=False):
26     feats = group[FEATURE_COLUMNS].values.astype(np.float32)
27     if has_labels:
28         labels = group["label"].map(label_to_idx).fillna(0).values
29     else:
30         labels = np.zeros(len(group), dtype=np.int64)
31     sids = group["session_id"].values
32     n = len(group)
33     for idx in range(n):
34         start = max(0, idx - window + 1)
35         chunk = feats[start:idx + 1]
36         if len(chunk) < window:
37             pad = np.zeros((window - len(chunk), n_features), dtype=np.float32)
38             chunk = np.vstack([pad, chunk])
39         sequences.append(chunk)
40         targets.append(labels[idx])
41         session_ids.append(sids[idx])
42
43 x = np.stack(sequences)
44 y = np.array(targets, dtype=np.int64)
45 return x, y, np.array(session_ids)
46
47
48 class SessionGRU(nn.Module):
49     def __init__(self, input_size, hidden_size=48, n_classes=len(ALL_LABELS)):
50         super().__init__()
51         self.gru = nn.GRU(input_size, hidden_size, num_layers=1, batch_first=True)
52         self.head = nn.Sequential(
53             nn.Linear(hidden_size, 32),
54             nn.ReLU(),
55             nn.Linear(32, n_classes),
56         )
57
58     def forward(self, x):
59         _, h = self.gru(x)
60         return self.head(h[-1])
61
62
63 class SequenceAnomalyModel:
64     def __init__(self, window=WINDOW, hidden_size=48, epochs=10, lr=1e-3, device="cpu"):
65         self.window = window
66         self.hidden_size = hidden_size
67         self.epochs = epochs
68         self.lr = lr
69         self.device = device

```

```

70     self.scaler = None
71     self.model = None
72
73     def _scale(self, x):
74         n, w, f = x.shape
75         flat = x.reshape(-1, f)
76         flat = self.scaler.transform(flat)
77         return flat.reshape(n, w, f).astype(np.float32)
78
79     def fit(self, x_train, y_train):
80         flat = x_train.reshape(-1, x_train.shape[-1])
81         self.scaler = StandardScaler().fit(flat)
82         x_scaled = self._scale(x_train)
83
84         class_counts = np.bincount(y_train, minlength=len(ALL_LABELS)).astype(np.float32)
85         class_counts[class_counts == 0] = 1.0
86         weights = (1.0 / class_counts)
87         weights = weights / weights.sum() * len(ALL_LABELS)
88
89         self.model = SessionGRU(input_size=x_train.shape[-1], hidden_size=self.hidden_size)
90         optimizer = torch.optim.Adam(self.model.parameters(), lr=self.lr)
91         criterion = nn.CrossEntropyLoss(weight=torch.tensor(weights))
92
93         x_t = torch.tensor(x_scaled)
94         y_t = torch.tensor(y_train)
95
96         n = len(x_t)
97         batch_size = 256
98         self.model.train()
99         for epoch in range(self.epochs):
100             perm = torch.randperm(n)
101             total_loss = 0.0
102             for i in range(0, n, batch_size):
103                 idx = perm[i:i + batch_size]
104                 optimizer.zero_grad()
105                 out = self.model(x_t[idx])
106                 loss = criterion(out, y_t[idx])
107                 loss.backward()
108                 optimizer.step()
109                 total_loss += loss.item() * len(idx)
110             return self
111
112     def predict_proba(self, x):
113         self.model.eval()
114         x_scaled = self._scale(x)
115         with torch.no_grad():
116             logits = self.model(torch.tensor(x_scaled))
117             probs = torch.softmax(logits, dim=1).numpy()
118         return probs

```

src/explain/attribution.py

```

1  import json
2
3  import numpy as np
4  import pandas as pd
5  import shap

```

```

6
7 from src.explain.narrative import compose_sentence
8 from src.features.build_features import FEATURE_COLUMNS
9 from src.models.cold_start import cold_start_note
10
11
12 def build_explainer(tabular_classifier):
13     return shap.TreeExplainer(tabular_classifier.model)
14
15
16 def _class_index(classes, label):
17     return classes.index(label) if label in classes else 0
18
19
20 def top_contributions(explainer, features_df, predicted_labels, classes, top_k=3):
21     """Per row, the top_k features pushing the score toward the predicted class."""
22     values = explainer(features_df[FEATURE_COLUMNS].values).values
23
24     per_row = []
25     for i in range(len(features_df)):
26         if values.ndim == 3:
27             row_shap = values[i, :, _class_index(classes, predicted_labels[i])]
28         else:
29             row_shap = values[i]
30
31         order = np.argsort(-row_shap)[:top_k]
32         contributions = [
33             (FEATURE_COLUMNS[j], features_df.iloc[i][FEATURE_COLUMNS[j]], float(row_shap[j]))
34             for j in order
35             if row_shap[j] > 0
36         ]
37         per_row.append(contributions)
38     return per_row
39
40
41 def explain_sessions(explainer, features_df, predicted_labels, classes, top_k=3):
42     """Readable sentence per alert, plus the raw SHAP values behind it."""
43     per_row = top_contributions(explainer, features_df, predicted_labels, classes, top_k)
44
45     explanations, reasons, details = [], [], []
46     for i, contributions in enumerate(per_row):
47         row = features_df.iloc[i]
48         note = cold_start_note(row["entity_history_length"])
49         pairs = [(feat, value) for feat, value, _ in contributions]
50
51         explanations.append(compose_sentence(predicted_labels[i], pairs, row, note))
52         reasons.append("; ".join(feat for feat, _, _ in contributions))
53         details.append(json.dumps([
54             {"feature": feat, "value": float(value), "shap": shap_value}
55             for feat, value, shap_value in contributions
56         ]))
57
58     return pd.DataFrame({
59         "explanation": explanations,
60         "reasons": reasons,
61         "shap_detail": details,
62     })

```

src/explain/narrative.py

```
1  """Turns the top contributing features for an alert into one readable sentence.
2
3  Deterministic templating rather than a generation call: this runs for every
4  alert in the queue, and an analyst reading the same alert twice should get the
5  same wording.
6  """
7
8  TYPE_PHRASING = {
9      "brute_force": "brute force",
10     "credential_stuffing": "credential stuffing",
11     "impossible_travel": "impossible travel",
12     "lateral_movement": "lateral movement",
13     "device_spoofing": "device spoofing",
14     "insider_drift": "insider drift",
15     "low_and_slow_exfil": "low-and-slow exfiltration",
16 }
17
18
19 def _fmt_int(value):
20     return f"{int(round(value))},}"
21
22
23 # SHAP can push a feature toward the predicted class even when that feature's
24 own value is unremarkable (trees split on low values too). Showing an analyst
25 # "a 0% failure rate" as grounds for an alert is worse than showing nothing, so
26 # each phrase only renders above a value that is actually worth reading.
27 def _phrase_geo_velocity(value, row):
28     if value < 100:
29         return None
30     return f"an implied travel speed of {_fmt_int(value)} km/h since the previous session"
31
32
33 def _phrase_geo_distance_home(value, row):
34     if value < 100:
35         return None
36     city = row.get("geo_city")
37     where = f" ({city})" if isinstance(city, str) and city and city != "unknown-transit" else ""
38     return f"a login {_fmt_int(value)} km from this entity's usual location{where}"
39
40
41 def _phrase_new_resource(value, row):
42     if not value:
43         return None
44     return f"first-ever access to {row.get('resource_accessed', 'this resource')} by this entity"
45
46
47 def _phrase_device_mismatch(value, row):
48     if not value:
49         return None
50     os_name = row.get("device_os")
51     detail = f" (now reporting {os_name})" if isinstance(os_name, str) and os_name else ""
52     return f"a device fingerprint that does not match this entity's prior sessions{detail}"
53
54
55 def _phrase_ip_distinct_entities(value, row):
```

```

56     if value < 2:
57         return None
58     return f"a source IP being used against {_fmt_int(value)} other accounts in the same 10-minute
59 window"
60
61
62 def _phrase_ip_failure_rate(value, row):
63     if value < 0.2:
64         return None
65     return f"a {value:.0%} authentication failure rate from this source IP"
66
67
68 def _phrase_ip_session_count(value, row):
69     if value < 5:
70         return None
71     return f"{_fmt_int(value)} sessions from this source IP in 10 minutes"
72
73
74 def _phrase_entity_failed(value, row):
75     if value < 2:
76         return None
77     return f"{_fmt_int(value)} failed logins for this entity within 10 minutes"
78
79
80 def _phrase_resource_breadth(value, row):
81     if value < 8:
82         return None
83     return f"{_fmt_int(value)} different resources touched in 24 hours, well above this entity's
84 norm"
85
86
87 def _phrase_resource_breadth_7d(value, row):
88     if value < 6:
89         return None
90     return f"{_fmt_int(value)} different resources gradually touched over the trailing week"
91
92
93 def _phrase_hour_z(value, row):
94     if abs(value) < 1.5:
95         return None
96     direction = "later" if value > 0 else "earlier"
97     return f"a login {abs(value):.1f} standard deviations {direction} than this entity's usual
98 hours"
99
100
101 def _phrase_duration_z(value, row):
102     if abs(value) < 1.5:
103         return None
104     direction = "longer" if value > 0 else "shorter"
105     return f"a session {abs(value):.1f} standard deviations {direction} than this entity's norm"
106
107
108 def _phrase_privileged_command(value, row):
109     if not value:
110         return None
111     return "privileged commands executed during the session"
112

```

```

113
114 def _phrase_command_length(value, row):
115     if value < 5:
116         return None
117     return f"an unusually long command sequence ({_fmt_int(value)} actions)"
118
119
120 def _phrase_auth_failed(value, row):
121     if not value:
122         return None
123     return "a failed authentication on this session"
124
125
126 def _phrase_idle_gap(value, row):
127     if value >= 9999:
128         return "no prior activity on record for this entity"
129     if value < 72:
130         return None
131     return f"a {_fmt_int(value)}-hour gap since this entity was last active"
132
133
134 PHRASE_BUILDERS = {
135     "geo_velocity_kmh": _phrase_geo_velocity,
136     "geo_distance_from_home_km": _phrase_geo_distance_home,
137     "is_new_resource": _phrase_new_resource,
138     "is_device_mismatch": _phrase_device_mismatch,
139     "ip_distinct_entities_10min": _phrase_ip_distinct_entities,
140     "ip_failure_rate_10min": _phrase_ip_failure_rate,
141     "ip_session_count_10min": _phrase_ip_session_count,
142     "entity_failed_count_10min": _phrase_entity_failed,
143     "entity_resource_breadth_24h": _phrase_resource_breadth,
144     "entity_resource_breadth_7d": _phrase_resource_breadth_7d,
145     "hour_of_day_z": _phrase_hour_z,
146     "session_duration_z": _phrase_duration_z,
147     "has_privileged_command": _phrase_privileged_command,
148     "command_sequence_length": _phrase_command_length,
149     "auth_failed": _phrase_auth_failed,
150     "hours_since_last_session": _phrase_idle_gap,
151 }
152
153
154 def build_phrases(contributions, row):
155     phrases = []
156     for feature, value in contributions:
157         builder = PHRASE_BUILDERS.get(feature)
158         if builder is None:
159             continue
160         phrase = builder(value, row)
161         if phrase:
162             phrases.append(phrase)
163     return phrases
164
165
166 def _join_phrases(phrases):
167     if len(phrases) == 1:
168         return phrases[0]
169     if len(phrases) == 2:

```

```

170     return f"{phrases[0]}, combined with {phrases[1]}"
171     return f"{phrases[0]}, combined with {phrases[1]} and {phrases[2]}"
172
173
174 def compose_sentence(predicted_type, contributions, row, cold_start_note_text=None):
175     phrases = build_phrases(contributions, row)
176
177     if not phrases and predicted_type in TYPE_PHRASING:
178         base = (
179             f"Flagged as {TYPE_PHRASING[predicted_type]} on a combination of weak signals, "
180             "with no single feature standing out - see the SHAP breakdown"
181         )
182     elif not phrases:
183         base = "Flagged by the ensemble, but no single feature dominated the score"
184     elif predicted_type in TYPE_PHRASING:
185         base = f"Flagged as {TYPE_PHRASING[predicted_type]} due to {_join_phrases(phrases)}"
186     else:
187         base = f"Elevated risk with no clear attack pattern, driven by {_join_phrases(phrases)}"
188
189     sentence = base + "."
190     if cold_start_note_text:
191         sentence += f" {cold_start_note_text}"
192     return sentence

```

src/dashboard/app.py

```

1  import json
2  from pathlib import Path
3
4  import pandas as pd
5  import streamlit as st
6
7  from src.dashboard import feedback
8
9  ROOT = Path(__file__).resolve().parents[2]
10 PROCESSED_DIR = ROOT / "data" / "processed"
11 FIGURE_DIR = ROOT / "reports" / "figures"
12
13 st.set_page_config(page_title="Behavioral Anomaly Detection", layout="wide")
14
15
16 @st.cache_data
17 def load_data():
18     scored = pd.read_csv(
19         PROCESSED_DIR / "scored_sessions.csv", parse_dates=["timestamp"], low_memory=False
20     )
21     with open(PROCESSED_DIR / "metrics.json") as f:
22         metrics = json.load(f)
23     return scored, metrics
24
25
26 @st.cache_data
27 def load_demo_summary():
28     path = PROCESSED_DIR / "demo_coldstart_drift.json"
29     if not path.exists():
30         return None
31     with open(path) as f:

```



```

89         round(alert["tabular_anomaly"], 3),
90         round(alert["sequence_anomaly"], 3),
91     ],
92 },
93     use_container_width=True, hide_index=True,
94 )
95
96 with st.expander("Raw SHAP contributions"):
97     detail = alert.get("shap_detail")
98     if isinstance(detail, str) and detail.strip():
99         st.dataframe(pd.DataFrame(json.loads(detail)), use_container_width=True,
100                          hide_index=True)
101     else:
102         st.caption("No attribution stored for this session.")
103
104 st.markdown("***Triage**")
105 b1, b2, _ = st.columns([1, 1, 4])
106 if b1.button("Confirm threat", type="primary", use_container_width=True):
107     feedback.record_decision(alert["session_id"], alert["entity_id"], feedback.CONFIRM)
108     st.rerun()
109 if b2.button("Dismiss as false positive", use_container_width=True):
110     feedback.record_decision(alert["session_id"], alert["entity_id"], feedback.DISMISS)
111     st.rerun()
112
113
114 def alert_queue_tab(scored, decisions):
115     c1, c2, c3 = st.columns(3)
116     entity_types = c1.multiselect(
117         "Entity type", sorted(scored["entity_type"].unique()),
118         default=list(scored["entity_type"].unique()),
119     )
120     pred_types = c2.multiselect(
121         "Predicted type", sorted(scored["predicted_type"].unique()),
122         default=[t for t in sorted(scored["predicted_type"].unique()) if t != "normal"],
123     )
124     budget_only = c3.checkbox("Top 1% risk budget only", value=True)
125
126     filtered = scored[scored["entity_type"].isin(entity_types)]
127     if pred_types:
128         filtered = filtered[filtered["predicted_type"].isin(pred_types)]
129     if budget_only:
130         filtered = filtered[filtered["risk_score"] >= scored["risk_score"].quantile(0.99)]
131
132     decided = set(decisions["session_id"]) if not decisions.empty else set()
133     if decided and st.checkbox("Hide sessions I've already triaged", value=True):
134         filtered = filtered[~filtered["session_id"].isin(decided)]
135
136     filtered = filtered.sort_values("effective_risk", ascending=False).head(300)
137     st.caption(f"{len(filtered)} alerts shown, ranked by risk after analyst feedback")
138
139     if filtered.empty:
140         st.success("Nothing left in the queue for these filters.")
141         return
142
143     selection = st.dataframe(
144         filtered[[
145             "entity_id", "entity_type", "timestamp", "predicted_type",

```

```

146         "effective_risk", "risk_adjustment", "confidence", "resource_accessed", "explanation",
147     ]],
148     use_container_width=True, height=340, hide_index=True,
149     on_select="rerun", selection_mode="single-row",
150 )
151
152 rows = selection["selection"]["rows"]
153 st.divider()
154 if not rows:
155     st.caption("Select a row above to open the alert.")
156     return
157 alert_detail(filtered.iloc[rows[0]], scored)
158
159
160 def system_behavior_tab(demo):
161     st.markdown(
162         "How the system handles entities with no history, and behaviour that "
163         "legitimately changes over time. Both charts are regenerated by "
164         "`python -m src.demo.coldstart_drift`."
165     )
166
167     cold_start_png = FIGURE_DIR / "cold_start_progression.png"
168     drift_png = FIGURE_DIR / "drift_vs_attack.png"
169
170     if not cold_start_png.exists():
171         st.warning("Run `python -m src.demo.coldstart_drift` to generate these charts.")
172         return
173
174     st.markdown("#### Cold start")
175     st.image(str(cold_start_png), use_container_width=True)
176     if demo:
177         checkpoints = pd.DataFrame(demo["cold_start"]["checkpoints"])
178         st.dataframe(
179             checkpoints.rename(columns={
180                 "session": "session #",
181                 "mean_blended_risk": "reported risk",
182                 "mean_raw_risk": "raw model risk",
183                 "std_across_devices": "sd across devices",
184                 "blend_weight_on_own_history": "weight on own history",
185             }).drop(columns=["n_devices"]),
186             use_container_width=True, hide_index=True,
187         )
188         first = demo["cold_start"]["checkpoints"][0]
189         st.caption(
190             f"A brand-new device scores {first['mean_raw_risk']:.2f} from the models alone - "
191             f"it is novel on every feature. Blending against the population baseline reports "
192             f"{first['mean_blended_risk']:.2f} instead, which is what keeps new devices out of "
193             "the alert queue."
194         )
195
196     st.markdown("#### Concept drift")
197     st.image(str(drift_png), use_container_width=True)
198     if demo:
199         mech = demo["drift_mechanism"]
200         pop = demo["drift_vs_attack"]["population"]
201         st.caption(
202             f"Left: after a permanent change in behaviour, the trailing "

```

```

203         f"{mech['rolling_window']}] baseline settles back to |z| "
204         f"{mech['rolling_z_once_settled']:.2f}, while a baseline that is never refreshed "
205         f"stays at {mech['frozen_baseline_z_once_settled']:.1f} forever. "
206         f"Right: this only covers timing and duration. Resource-footprint growth is not "
207         f"forgiven - median risk rose {pop['median_risk_before']:.2f} to "
208         f"{pop['median_risk_during_shift']:.2f} across "
209         f"{pop['n_drift_entities_considered']} drifting entities, though none of them "
210         "crossed the top-1% alert threshold."
211     )
212
213
214 def feedback_tab(decisions, scored):
215     st.markdown(
216         "Dismissals push an entity down the queue and confirmations pull it up, by "
217         f"{feedback.PERCENTILE_STEP:.2f} percentile ranks per decision, capped at "
218         f"**{feedback.PERCENTILE_CAP:.2f} percentile ranks** however many decisions "
219         "are recorded. Because the top-1% queue spans exactly one percentile point, "
220         f"that cap means an entity can be moved through at most "
221         f"{feedback.PERCENTILE_CAP * 100:.0f}% of the queue. This only reorders the "
222         "queue - it does not retrain the models or change any stored risk score."
223     )
224     st.caption(
225         "The earlier version capped the raw score offset at "
226         f"+/-{feedback.MAX_STEPS * feedback.DISMISS_STEP:.2f} instead. That bounded "
227         "far less than it looked like: scores cluster tightly just above the cutoff, "
228         "so 0.10 of score was wider than the cutoff margin for 96% of queued entities."
229     )
230
231     if decisions.empty:
232         st.info("No decisions recorded yet. Confirm or dismiss an alert to start.")
233         return
234
235     c1, c2 = st.columns(2)
236     with c1:
237         st.markdown("***Decisions***")
238         st.dataframe(
239             decisions.sort_values("decided_at", ascending=False),
240             use_container_width=True, height=280, hide_index=True,
241         )
242     with c2:
243         st.markdown("***Current per-entity movement***")
244         steps = feedback.entity_net_steps(decisions)
245         moved = (
246             scored[scored["entity_id"].isin(steps)]
247             .groupby("entity_id")["pct_movement"].max()
248             .to_dict()
249         )
250         st.dataframe(
251             pd.DataFrame([
252                 {
253                     "entity_id": eid,
254                     "net decisions": net,
255                     "percentile movement": round(moved.get(eid, 0.0), 3),
256                     "at cap": abs(moved.get(eid, 0.0)) >= feedback.PERCENTILE_CAP - 1e-9,
257                 }
258                 for eid, net in steps.items()
259             ]).sort_values("percentile movement"),

```

```

260         use_container_width=True, height=280, hide_index=True,
261     )
262
263
264 def main():
265     st.title("Behavioral Anomaly Detection - Analyst Dashboard")
266
267     scored, metrics = load_data()
268     decisions = feedback.load_decisions()
269     scored = feedback.apply_feedback(scored, decisions)
270
271     top_level_metrics(scored, metrics, decisions)
272     st.divider()
273
274     queue_tab, behavior_tab, review_tab = st.tabs(
275         ["Alert queue", "System behaviour", "Analyst feedback"]
276     )
277     with queue_tab:
278         alert_queue_tab(scored, decisions)
279     with behavior_tab:
280         system_behavior_tab(load_demo_summary())
281     with review_tab:
282         feedback_tab(decisions, scored)
283
284
285 if __name__ == "__main__":
286     main()

```

src/dashboard/feedback.py

```

1  """Analyst triage decisions and the per-entity adjustment they produce.
2
3  The adjustment is deliberately dumb: a fixed step per decision, capped, applied
4  only to alert ranking. It is not model retraining and does not change any
5  stored risk score - it just pushes entities an analyst keeps dismissing further
6  down the queue, and pulls confirmed ones up.
7
8  Two capping modes exist:
9
10 - ``score`` (the original) caps the raw score offset at +/-0.10. This turned out
11 to bound almost nothing: risk scores cluster tightly just above the alert
12 cutoff, so 0.10 of score is wider than the cutoff margin for 96% of queued
13 entities and a handful of dismissals can clear nearly any entity out of the
14 queue entirely.
15 - ``percentile`` (the default) caps how far an entity can move in *percentile
16 rank* instead. Because the top-1% queue spans exactly one percentile point, a
17 cap of N points means precisely "an entity can be moved through N% of the
18 queue" - so the cap value states what it actually enforces, independent of how
19 tightly scores happen to be clustered.
20
21 Both are kept so the report can compare them on the same data.
22 """
23
24 import sqlite3
25 from datetime import datetime, timezone
26 from pathlib import Path
27

```

```

28 import numpy as np
29 import pandas as pd
30
31 ROOT = Path(__file__).resolve().parents[2]
32 DB_PATH = ROOT / "data" / "processed" / "analyst_feedback.db"
33
34 DISMISS_STEP = 0.02
35 CONFIRM_STEP = 0.02
36 MAX_STEPS = 5
37
38 # Percentile-rank capping. PERCENTILE_STEP * MAX_STEPS == PERCENTILE_CAP, so a
39 # decision still "counts" the same way and five of them still max out the
40 # adjustment - only the ceiling is expressed in queue movement rather than score.
41 PERCENTILE_STEP = 0.05
42 PERCENTILE_CAP = 0.25
43
44 CAP_MODE_SCORE = "score"
45 CAP_MODE_PERCENTILE = "percentile"
46 DEFAULT_CAP_MODE = CAP_MODE_PERCENTILE
47
48 CONFIRM = "confirm"
49 DISMISS = "dismiss"
50
51
52 def connect(db_path=DB_PATH):
53     db_path = Path(db_path)
54     db_path.parent.mkdir(parents=True, exist_ok=True)
55     conn = sqlite3.connect(db_path)
56     conn.execute(
57         """
58         CREATE TABLE IF NOT EXISTS decisions (
59             session_id TEXT PRIMARY KEY,
60             entity_id TEXT NOT NULL,
61             decision TEXT NOT NULL,
62             decided_at TEXT NOT NULL
63         )
64         """
65     )
66     conn.commit()
67     return conn
68
69
70 def record_decision(session_id, entity_id, decision, db_path=DB_PATH):
71     if decision not in (CONFIRM, DISMISS):
72         raise ValueError(f"unknown decision: {decision}")
73
74     conn = connect(db_path)
75     try:
76         conn.execute(
77             """
78             INSERT INTO decisions (session_id, entity_id, decision, decided_at)
79             VALUES (?, ?, ?, ?)
80             ON CONFLICT(session_id) DO UPDATE SET
81                 decision = excluded.decision,
82                 decided_at = excluded.decided_at
83             """
84             ,
85             (session_id, entity_id, decision, datetime.now(timezone.utc).isoformat()),

```

```

85         )
86         conn.commit()
87     finally:
88         conn.close()
89
90
91 def load_decisions(db_path=DB_PATH):
92     if not Path(db_path).exists():
93         return pd.DataFrame(columns=["session_id", "entity_id", "decision", "decided_at"])
94     conn = connect(db_path)
95     try:
96         return pd.read_sql_query("SELECT * FROM decisions", conn)
97     finally:
98         conn.close()
99
100
101 def entity_adjustments(decisions):
102     """Net score offset per entity, negative meaning "rank this lower"."""
103     if decisions.empty:
104         return {}
105
106     counts = decisions.groupby(["entity_id", "decision"]).size().unstack(fill_value=0)
107
108     adjustments = {}
109     for entity_id, row in counts.iterrows():
110         offset = (
111             CONFIRM_STEP * min(int(row.get(CONFIRM, 0)), MAX_STEPS)
112             - DISMISS_STEP * min(int(row.get(DISMISS, 0)), MAX_STEPS)
113         )
114         if offset:
115             adjustments[entity_id] = offset
116     return adjustments
117
118
119 def apply_adjustments(scored, adjustments):
120     """Legacy score-space capping. Kept for comparison against percentile mode."""
121     result = scored.copy()
122     result["risk_adjustment"] = result["entity_id"].map(adjustments).fillna(0.0)
123     result["effective_risk"] = (result["risk_score"] + result["risk_adjustment"]).clip(0, 1)
124     return result
125
126
127 def entity_net_steps(decisions):
128     """Net decision count per entity, positive meaning "rank this higher".
129
130     Deliberately uncapped - in percentile mode the bound is applied to queue
131     movement, not to how many times an analyst is allowed to click.
132     """
133     if decisions.empty:
134         return {}
135
136     counts = decisions.groupby(["entity_id", "decision"]).size().unstack(fill_value=0)
137
138     steps = {}
139     for entity_id, row in counts.iterrows():
140         net = int(row.get(CONFIRM, 0)) - int(row.get(DISMISS, 0))
141         if net:

```

```

142         steps[entity_id] = net
143     return steps
144
145
146 def percentile_ranks(scores):
147     """Percentile rank (0-100) of each score within its own distribution.
148
149     Normalised by ``n - 1`` rather than ``n`` so that feeding a rank straight
150     back through ``np.quantile`` returns the original score exactly - i.e. zero
151     movement is genuinely a no-op rather than a small resampling nudge.
152     """
153     scores = np.asarray(scores, dtype=float)
154     n = len(scores)
155     if n == 0:
156         return np.array([])
157     if n == 1:
158         return np.zeros(1)
159     ranks = np.empty(n)
160     ranks[np.argsort(scores, kind="mergesort")] = np.arange(n)
161     return ranks / (n - 1) * 100.0
162
163
164 def apply_percentile_adjustments(scored, net_steps, cap=PERCENTILE_CAP,
165                                 step=PERCENTILE_STEP):
166     """Move each entity's alerts by at most ``cap`` percentile ranks.
167
168     The requested movement is ``net_steps * step`` percentile points, clipped to
169     +/-cap however many decisions were recorded, then mapped back through the
170     empirical score distribution so the queue can still be ranked on a score.
171     """
172     result = scored.copy()
173     scores = result["risk_score"].to_numpy(dtype=float)
174
175     pct = percentile_ranks(scores)
176     requested = result["entity_id"].map(net_steps).fillna(0.0).to_numpy(dtype=float) * step
177     movement = np.clip(requested, -cap, cap)
178     target_pct = np.clip(pct + movement, 0.0, 100.0)
179
180     effective = np.quantile(scores, target_pct / 100.0)
181
182     result["pct_rank"] = pct
183     result["pct_movement"] = target_pct - pct
184     result["effective_pct_rank"] = target_pct
185     result["effective_risk"] = np.clip(effective, 0.0, 1.0)
186     result["risk_adjustment"] = result["effective_risk"] - result["risk_score"]
187     return result
188
189
190 def apply_feedback(scored, decisions, mode=DEFAULT_CAP_MODE, cap=None):
191     """Apply analyst decisions to the queue under the chosen capping mode.
192
193     Both modes return ``risk_adjustment`` and ``effective_risk`` so callers do
194     not need to care which one is active.
195     """
196     if mode == CAP_MODE_SCORE:
197         return apply_adjustments(scored, entity_adjustments(decisions))
198     if mode == CAP_MODE_PERCENTILE:

```

```

199         return apply_percentile_adjustments(
200             scored, entity_net_steps(decisions),
201             cap=PERCENTILE_CAP if cap is None else cap,
202         )
203     raise ValueError(f"unknown cap mode: {mode}")

```

src/pipeline.py

```

1  import json
2  from pathlib import Path
3
4  import numpy as np
5  import pandas as pd
6  from sklearn.metrics import average_precision_score, classification_report
7
8  from src.explain.attribution import build_explainer, explain_sessions
9  from src.features.build_features import build_features
10 from src.generator.config import ALL_LABELS, SIM_DAYS
11 from src.generator.generate_dataset import build_dataset
12 from src.models.baseline import BaselineProfiler
13 from src.models.classifier import TabularClassifier
14 from src.models.cold_start import blend_cold_start_scores
15 from src.models.persistence import save_bundle
16 from src.models.sequence_model import SequenceAnomalyModel, build_sequences
17
18 ROOT = Path(__file__).resolve().parents[1]
19 RAW_DIR = ROOT / "data" / "raw"
20 PROCESSED_DIR = ROOT / "data" / "processed"
21 REPORTS_DIR = ROOT / "reports"
22
23 TRAIN_FRACTION = 0.7
24 RISK_WEIGHTS = {"baseline": 0.25, "tabular": 0.35, "sequence": 0.40}
25 ALERT_BUDGET = 0.01
26 REASON_COVERAGE = 0.05
27
28
29 def load_or_generate_raw(seed):
30     access_path = RAW_DIR / "access_log.csv"
31     labels_path = RAW_DIR / "labels.csv"
32     entities_path = RAW_DIR / "entities.csv"
33
34     if access_path.exists() and labels_path.exists() and entities_path.exists():
35         access_log = pd.read_csv(access_path, parse_dates=["timestamp"])
36         labels = pd.read_csv(labels_path)
37         entities = pd.read_csv(entities_path)
38         return access_log, labels, entities
39
40     RAW_DIR.mkdir(parents=True, exist_ok=True)
41     df, entities = build_dataset(seed)
42     labels = df[["session_id", "label"]].copy()
43     from src.generator.generate_dataset import COLUMN_ORDER
44     access_log = df[COLUMN_ORDER].copy()
45
46     access_log.to_csv(access_path, index=False)
47     labels.to_csv(labels_path, index=False)
48     entities.to_csv(entities_path, index=False)
49     return access_log, labels, entities

```

```

50
51
52 def time_split_mask(timestamps, train_fraction=TRAIN_FRACTION):
53     cutoff_day = int(SIM_DAYS * train_fraction)
54     cutoff_ts = timestamps.min() + pd.Timedelta(days=cutoff_day)
55     return timestamps < cutoff_ts
56
57
58 def train_models(train_df):
59     baseline = BaselineProfiler().fit(train_df)
60     tabular = TabularClassifier().fit(train_df)
61
62     x_seq, y_seq, seq_session_ids = build_sequences(train_df)
63     seq_model = SequenceAnomalyModel(epochs=8).fit(x_seq, y_seq)
64
65     # the cold-start prior is fixed on training data so that scoring a new
66     # entity later doesn't depend on whatever else happens to be in the batch.
67     train_components = _component_scores(train_df, baseline, tabular, seq_model)
68     type_prior_map = (
69         pd.Series(train_components["raw_risk"], index=train_df["entity_type"])
70         .groupby(level=0).mean().to_dict()
71     )
72
73     return {
74         "baseline": baseline,
75         "tabular": tabular,
76         "sequence": seq_model,
77         "type_prior_map": type_prior_map,
78     }
79
80
81 def _component_scores(features, baseline, tabular, seq_model):
82     baseline_risk = baseline.score(features)
83
84     tabular_proba = tabular.predict_proba(features)
85     tabular_classes = tabular.classes_
86     tabular_anomaly = 1 - tabular_proba[:, tabular_classes.index("normal")]
87
88     x_seq, _, seq_session_ids = build_sequences(features)
89     seq_order = pd.Series(range(len(seq_session_ids)), index=seq_session_ids)
90     x_seq = x_seq[seq_order.loc[features["session_id"]].values]
91     seq_proba = seq_model.predict_proba(x_seq)
92     seq_anomaly = 1 - seq_proba[:, ALL_LABELS.index("normal")]
93
94     tabular_proba_aligned = np.zeros((len(features), len(ALL_LABELS)))
95     for j, cls in enumerate(tabular_classes):
96         tabular_proba_aligned[:, ALL_LABELS.index(cls)] = tabular_proba[:, j]
97     combined_proba = (tabular_proba_aligned + seq_proba) / 2
98
99     raw_risk = (
100         RISK_WEIGHTS["baseline"] * baseline_risk
101         + RISK_WEIGHTS["tabular"] * tabular_anomaly
102         + RISK_WEIGHTS["sequence"] * seq_anomaly
103     )
104
105     return {
106         "baseline_risk": baseline_risk,

```

```

107         "tabular_anomaly": tabular_anomaly,
108         "sequence_anomaly": seq_anomaly,
109         "combined_proba": combined_proba,
110         "raw_risk": raw_risk,
111     }
112
113
114 def score_frame(features, models):
115     components = _component_scores(
116         features, models["baseline"], models["tabular"], models["sequence"]
117     )
118     combined_proba = components["combined_proba"]
119     predicted_idx = combined_proba.argmax(axis=1)
120
121     risk_score = blend_cold_start_scores(
122         features, components["raw_risk"], models["type_prior_map"]
123     )
124
125     return {
126         "risk_score": risk_score,
127         "raw_risk": components["raw_risk"],
128         "predicted_type": [ALL_LABELS[i] for i in predicted_idx],
129         "confidence": combined_proba.max(axis=1),
130         "baseline_risk": components["baseline_risk"],
131         "tabular_anomaly": components["tabular_anomaly"],
132         "sequence_anomaly": components["sequence_anomaly"],
133     }
134
135
136 def run(seed=42):
137     access_log, labels, entities = load_or_generate_raw(seed)
138     features = build_features(access_log)
139     features = features.merge(labels, on="session_id", how="left")
140
141     train_mask = time_split_mask(features["timestamp"])
142     train_df = features[train_mask].reset_index(drop=True)
143
144     models = train_models(train_df)
145     scores = score_frame(features, models)
146     risk_score = scores["risk_score"]
147     predicted_type = scores["predicted_type"]
148
149     save_bundle(models)
150
151     # SHAP attribution is only worth the cost for sessions an analyst would
152     # actually see, not the entire (mostly normal) session log.
153     reason_cutoff = np.quantile(risk_score, 1 - REASON_COVERAGE)
154     reason_mask = risk_score >= reason_cutoff
155     explainer = build_explainer(models["tabular"])
156     explained = explain_sessions(
157         explainer,
158         features[reason_mask].reset_index(drop=True),
159         np.array(predicted_type)[reason_mask],
160         models["tabular"].classes_,
161     )
162
163     scored = features[[

```

```

164     "session_id", "entity_id", "entity_type", "timestamp", "resource_accessed",
165     "source_ip", "auth_success", "entity_history_length", "is_cold_start", "label",
166     ]).copy()
167     scored["risk_score"] = risk_score
168     scored["predicted_type"] = predicted_type
169     scored["confidence"] = scores["confidence"]
170     scored["baseline_risk"] = scores["baseline_risk"]
171     scored["tabular_anomaly"] = scores["tabular_anomaly"]
172     scored["sequence_anomaly"] = scores["sequence_anomaly"]
173
174     scored["explanation"] = ""
175     scored["reasons"] = ""
176     scored["shap_detail"] = ""
177     scored.loc[reason_mask, "explanation"] = explained["explanation"].values
178     scored.loc[reason_mask, "reasons"] = explained["reasons"].values
179     scored.loc[reason_mask, "shap_detail"] = explained["shap_detail"].values
180
181     scored = scored.sort_values("risk_score", ascending=False).reset_index(drop=True)
182
183     PROCESSED_DIR.mkdir(parents=True, exist_ok=True)
184     scored.to_csv(PROCESSED_DIR / "scored_sessions.csv", index=False)
185
186     metrics = evaluate(features["label"].values, predicted_type, risk_score, train_mask.values)
187     with open(PROCESSED_DIR / "metrics.json", "w") as f:
188         json.dump(metrics, f, indent=2)
189
190     return scored, metrics
191
192
193 def evaluate(true_labels, predicted_type, risk_score, train_mask):
194     test_mask = ~train_mask
195     y_true_test = true_labels[test_mask]
196     y_pred_test = np.array(predicted_type)[test_mask]
197
198     report = classification_report(y_true_test, y_pred_test, output_dict=True, zero_division=0)
199
200     is_anomaly = (true_labels != "normal").astype(int)
201     pr_auc = average_precision_score(is_anomaly[test_mask], risk_score[test_mask])
202
203     n_alerts = max(1, int(len(y_true_test) * ALERT_BUDGET))
204     order = np.argsort(-risk_score[test_mask])[:n_alerts]
205     top_alert_true = is_anomaly[test_mask][order]
206     precision_at_budget = top_alert_true.mean()
207     false_positive_rate_at_budget = 1 - precision_at_budget
208
209     return {
210         "classification_report": report,
211         "pr_auc_binary_anomaly": pr_auc,
212         "alert_budget_fraction": ALERT_BUDGET,
213         "precision_at_alert_budget": float(precision_at_budget),
214         "false_positive_rate_at_alert_budget": float(false_positive_rate_at_budget),
215         "n_test_sessions": int(test_mask.sum()),
216         "n_test_anomalies": int(is_anomaly[test_mask].sum()),
217     }
218
219
220 if __name__ == "__main__":

```

```

221     scored, metrics = run()
222     print(f"scored {len(scored)} sessions")
223     print(f"PR-AUC (binary anomaly): {metrics['pr_auc_binary_anomaly']:.3f}")
224     print(f"precision @ top {metrics['alert_budget_fraction']:.0%} alert budget: "
225           f"{metrics['precision_at_alert_budget']:.3f}")

```

src/demo/coldstart_drift.py

```

1  """Reproduces the cold-start and concept-drift evidence used in the report.
2
3  Run with `python -m src.demo.coldstart_drift` after the pipeline has trained
4 once. Writes two charts to reports/figures/ and the underlying numbers to
5 data/processed/demo_coldstart_drift.json so the report and dashboard both read
6 real output rather than restating claims.
7  """
8
9  import json
10 from pathlib import Path
11
12 import matplotlib
13 import numpy as np
14 import pandas as pd
15
16 matplotlib.use("Agg")
17 import matplotlib.pyplot as plt # noqa: E402
18
19 from src.features.build_features import COLD_START_THRESHOLD, build_features # noqa: E402
20 from src.features.drift import DRIFT_WINDOW, rolling_zscore # noqa: E402
21 from src.generator import config # noqa: E402
22 from src.generator.entities import build_entities # noqa: E402
23 from src.generator.sessions import generate_normal_sessions # noqa: E402
24 from src.models.persistence import load_bundle # noqa: E402
25 from src.pipeline import RAW_DIR, score_frame # noqa: E402
26
27 ROOT = Path(__file__).resolve().parents[2]
28 FIGURE_DIR = ROOT / "reports" / "figures"
29 PROCESSED_DIR = ROOT / "data" / "processed"
30
31 N_DEMO_DEVICES = 20
32 CHECKPOINTS = [1, 5, 20, 50]
33 MAX_SESSION_INDEX = 60
34 DEMO_SEED = 991
35
36
37 def load_raw():
38     access_log = pd.read_csv(RAW_DIR / "access_log.csv", parse_dates=["timestamp"])
39     labels = pd.read_csv(RAW_DIR / "labels.csv")
40     return access_log, labels
41
42
43 def make_demo_devices(n=N_DEMO_DEVICES, seed=DEMO_SEED):
44     """Fresh edge devices with no history in the trained population."""
45     profiles = build_entities(seed)
46     device_ids = [e for e, p in profiles.items() if p["entity_type"] == "edge_device"][:n]
47
48     demo_profiles = {}
49     for i, eid in enumerate(device_ids):

```

```

50     profile = dict(profiles[eid])
51     profile["joined_day"] = 0
52     profile["entity_id"] = f"demo_dev_{i:02d}"
53     demo_profiles[profile["entity_id"]] = profile
54
55     rows = generate_normal_sessions(demo_profiles, seed)
56     return pd.DataFrame(rows)
57
58
59 def cold_start_curve(access_log, models):
60     demo_rows = make_demo_devices()
61     demo_rows = demo_rows.drop(columns=["label"])
62
63     combined = pd.concat([access_log, demo_rows], ignore_index=True)
64     features = build_features(combined)
65     scores = score_frame(features, models)
66     features = features.assign(
67         risk_score=scores["risk_score"], raw_risk=scores["raw_risk"]
68     )
69
70     demo = features[features["entity_id"].str.startswith("demo_dev_")].copy()
71     demo = demo.sort_values(["entity_id", "timestamp"])
72     demo["session_index"] = demo.groupby("entity_id").cumcount() + 1
73     demo = demo[demo["session_index"] <= MAX_SESSION_INDEX]
74
75     by_index = demo.groupby("session_index").agg(
76         mean_risk=("risk_score", "mean"),
77         std_risk=("risk_score", "std"),
78         mean_raw=("raw_risk", "mean"),
79         n=("risk_score", "size"),
80     ).reset_index()
81
82     checkpoints = []
83     for cp in CHECKPOINTS:
84         row = by_index[by_index["session_index"] == cp]
85         if row.empty:
86             continue
87         row = row.iloc[0]
88         # spread of an individual device's score around the cohort mean at this depth
89         at_cp = demo[demo["session_index"] == cp]["risk_score"]
90         checkpoints.append({
91             "session": int(cp),
92             "mean_blended_risk": float(row["mean_risk"]),
93             "mean_raw_risk": float(row["mean_raw"]),
94             "std_across_devices": float(at_cp.std()),
95             "blend_weight_on_own_history": float(min(1.0, (cp - 1) / COLD_START_THRESHOLD)),
96             "n_devices": int(row["n"]),
97         })
98
99     return by_index, checkpoints, demo
100
101
102 def drift_mechanism_check():
103     """Direct check that the rolling baseline forgives a persistent shift.
104
105     A single entity whose session duration steps up permanently on day 15.
106     Compared against a baseline profiled once and never refreshed, which is the

```

```

107     failure mode the trailing window exists to avoid.
108     """
109     n_days = 60
110     step_day = 15
111     rng = np.random.default_rng(7)
112
113     timestamps = [config.SIM_START + pd.Timedelta(days=d) for d in range(n_days)]
114     values = np.where(
115         np.arange(n_days) < step_day,
116         rng.normal(300, 20, n_days),
117         rng.normal(900, 20, n_days),
118     )
119     frame = pd.DataFrame({
120         "entity_id": "drift_demo",
121         "timestamp": timestamps,
122         "session_duration": values,
123     })
124
125     rolling = rolling_zscore(frame, "session_duration").abs()
126
127     # baseline frozen on the pre-shift period, never updated afterwards
128     frozen_mean = values[:step_day].mean()
129     frozen_std = values[:step_day].std(ddof=1)
130     frozen = pd.Series(np.abs((values - frozen_mean) / frozen_std))
131
132     settled = np.arange(n_days) >= step_day + 30
133     return {
134         "step_day": step_day,
135         "rolling_window": DRIFT_WINDOW,
136         "rolling_z_peak_at_shift": float(rolling[step_day:step_day + 3].max()),
137         "rolling_z_once_settled": float(rolling[settled].mean()),
138         "frozen_baseline_z_once_settled": float(frozen[settled].mean()),
139         "series": {
140             "day": list(range(n_days)),
141             "rolling_z": [float(v) for v in rolling],
142             "frozen_z": [float(v) for v in frozen],
143         },
144     }
145
146
147 def _daily_risk(scored, entity_id):
148     """Peak, not mean: the queue ranks individual sessions, so a short burst
149     inside an otherwise quiet day is what decides whether an alert fires."""
150     rows = scored[scored["entity_id"] == entity_id].copy()
151     rows["day"] = rows["timestamp"].dt.floor("D")
152     return rows.groupby("day")["risk_score"].max()
153
154
155 def pick_drift_and_attack(scored):
156     """Legitimate slow-change entity vs a genuine attacker in the same window."""
157     drift_entities = scored[scored["label"] == "insider_drift"]["entity_id"].unique()
158
159     candidates = []
160     for eid in drift_entities:
161         rows = scored[scored["entity_id"] == eid]
162         drift_rows = rows[rows["label"] == "insider_drift"]
163         if len(drift_rows) < 5:

```

```

164         continue
165         shift_start = drift_rows["timestamp"].min()
166         before = rows[(rows["label"] == "normal") & (rows["timestamp"] < shift_start)]
167         if len(before) < 20:
168             continue
169         candidates.append({
170             "entity_id": eid,
171             "risk_before": float(before["risk_score"].mean()),
172             "risk_during_shift": float(drift_rows["risk_score"].mean()),
173             "shift_start": shift_start.isoformat(),
174         })
175
176         # the most favourable case for the drift story, so the report is not
177         # quoting a cherry-picked worst example
178         candidates.sort(key=lambda c: c["risk_during_shift"] - c["risk_before"])
179         drift_pick = candidates[0] if candidates else None
180
181         # rank attackers on their attack sessions only; averaging over an entity's
182         # whole history buries a short burst under months of normal traffic
183         attack_rows = scored[scored["label"].isin(
184             ["lateral_movement", "brute_force", "low_and_slow_exfil"]
185         )]
186         attack_stats = attack_rows.groupby("entity_id")["risk_score"].agg(["mean", "size"])
187         attack_stats = attack_stats[attack_stats["size"] >= 5].sort_values("mean", ascending=False)
188         attack_pick = attack_stats.index[0] if len(attack_stats) else None
189
190         deltas = [c["risk_during_shift"] - c["risk_before"] for c in candidates]
191         population = {
192             "n_drift_entities_considered": len(candidates),
193             "median_risk_before": float(np.median([c["risk_before"] for c in candidates])) if
194 candidates else None,
195             "median_risk_during_shift": float(np.median([c["risk_during_shift"] for c in candidates]))
196 if candidates else None,
197             "median_delta": float(np.median(deltas)) if deltas else None,
198             "n_entities_score_rose": int(sum(d > 0 for d in deltas)),
199         }
200         return drift_pick, attack_pick, candidates, population
201
202
203 def plot_cold_start(by_index, checkpoints, path):
204     fig, ax = plt.subplots(figsize=(8, 4.5))
205
206     x = by_index["session_index"]
207     mean = by_index["mean_risk"]
208     std = by_index["std_risk"].fillna(0)
209
210     ax.plot(x, mean, color="#1f77b4", label="blended risk score (what the system reports)")
211     ax.fill_between(x, mean - std, mean + std, color="#1f77b4", alpha=0.18,
212                    label="+/- 1 sd across the 20 new devices")
213     ax.plot(x, by_index["mean_raw"], color="#d62728", linestyle="--",
214            label="raw model score before cold-start blending")
215
216     ax.axvline(COLD_START_THRESHOLD, color="grey", linestyle=":", linewidth=1)
217     ax.annotate(f"cold-start blend fully released\nat {COLD_START_THRESHOLD} sessions",
218               xy=(COLD_START_THRESHOLD, ax.get_ylim()[1] * 0.92),
219               xytext=(COLD_START_THRESHOLD + 6, ax.get_ylim()[1] * 0.92),
220               fontsize=8, color="grey")

```

```

221
222     for cp in checkpoints:
223         ax.plot(cp["session"], cp["mean_blended_risk"], "o", color="#1f77b4", markersize=5)
224
225     ax.set_xlabel("session number for a brand-new device")
226     ax.set_ylabel("risk score")
227     ax.set_title("Cold start: how a new device's score settles as history accumulates")
228     ax.legend(fontsize=8, loc="upper right")
229     fig.tight_layout()
230     fig.savefig(path, dpi=130)
231     plt.close(fig)
232
233
234 def plot_drift(scored, drift_pick, attack_entity, mechanism, path):
235     fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(13, 4.6))
236
237     series = mechanism["series"]
238     ax1.plot(series["day"], series["rolling_z"], color="#1f77b4",
239             label=f"trailing {mechanism['rolling_window']} baseline (used here)")
240     ax1.plot(series["day"], series["frozen_z"], color="#d62728", linestyle="--",
241             label="baseline profiled once, never refreshed")
242     ax1.axvline(mechanism["step_day"], color="grey", linestyle=":", linewidth=1)
243     ax1.set_yscale("symlog")
244     ax1.set_ylim(bottom=0)
245     ax1.set_xlabel("day (permanent behaviour change on day 15)")
246     ax1.set_ylabel("|z| against the entity's own baseline")
247     ax1.set_title("Timing/duration drift is forgiven once it becomes the norm")
248     ax1.legend(fontsize=8)
249
250     drift_series = _daily_risk(scored, drift_pick["entity_id"])
251     ax2.plot(drift_series.index, drift_series.values, color="#2ca02c", marker="o", markersize=3,
252             label=f"{drift_pick['entity_id']} - legitimate resource-footprint growth")
253
254     attack_series = _daily_risk(scored, attack_entity)
255     ax2.plot(attack_series.index, attack_series.values, color="#d62728", marker="o", markersize=3,
256             label=f"{attack_entity} - genuine attack")
257
258     alert_cutoff = scored["risk_score"].quantile(0.99)
259     ax2.axhline(alert_cutoff, color="black", linestyle="--", linewidth=1,
260             label=f"top-1% alert threshold ({alert_cutoff:.2f})")
261     ax2.axvline(pd.Timestamp(drift_pick["shift_start"]), color="#2ca02c", linestyle=":",
262 linewidth=1)
263     ax2.set_xlabel("day")
264     ax2.set_ylabel("peak daily risk score")
265     ax2.set_title("Drift raises scores but stays under the alert bar")
266     ax2.legend(fontsize=8, loc="center right")
267
268     fig.autofmt_xdate()
269     fig.tight_layout()
270     fig.savefig(path, dpi=130)
271     plt.close(fig)
272
273
274 def main():
275     FIGURE_DIR.mkdir(parents=True, exist_ok=True)
276     PROCESSED_DIR.mkdir(parents=True, exist_ok=True)
277

```

```

278 models = load_bundle()
279 access_log, _ = load_raw()
280
281 by_index, checkpoints, _ = cold_start_curve(access_log, models)
282 plot_cold_start(by_index, checkpoints, FIGURE_DIR / "cold_start_progression.png")
283
284 scored = pd.read_csv(
285     PROCESSED_DIR / "scored_sessions.csv", parse_dates=["timestamp"], low_memory=False
286 )
287 mechanism = drift_mechanism_check()
288 drift_pick, attack_pick, candidates, population = pick_drift_and_attack(scored)
289 if drift_pick and attack_pick:
290     plot_drift(scored, drift_pick, attack_pick, mechanism, FIGURE_DIR / "drift_vs_attack.png")
291
292 attack_rows = scored[(scored["entity_id"] == attack_pick)
293                     & (scored["label"] != "normal")] if attack_pick else None
294 attack_risk = float(attack_rows["risk_score"].mean()) if attack_pick else None
295 summary = {
296     "cold_start": {
297         "n_devices": N_DEMO_DEVICES,
298         "threshold_sessions": COLD_START_THRESHOLD,
299         "checkpoints": checkpoints,
300     },
301     "drift_mechanism": {k: v for k, v in mechanism.items() if k != "series"},
302     "drift_vs_attack": {
303         "drift_entity": drift_pick,
304         "attack_entity": attack_pick,
305         "attack_mean_risk_on_attack_sessions": attack_risk,
306         "population": population,
307     },
308 }
309
310 with open(PROCESSED_DIR / "demo_coldstart_drift.json", "w") as f:
311     json.dump(summary, f, indent=2)
312
313 print("cold start (mean blended risk across new devices):")
314 for cp in checkpoints:
315     print(f" session {cp['session']:>3}: risk {cp['mean_blended_risk']:.3f} "
316           f"(raw {cp['mean_raw_risk']:.3f}, sd {cp['std_across_devices']:.3f}, "
317           f"own-history weight {cp['blend_weight_on_own_history']:.2f})")
318
319 print(f"\ndrift mechanism (permanent step change on day {mechanism['step_day']}):")
320 print(f" rolling z peak at the shift: {mechanism['rolling_z_peak_at_shift']:.2f}")
321 print(f" rolling z once settled: {mechanism['rolling_z_once_settled']:.2f}")
322 print(f" frozen-baseline z once settled: {mechanism['frozen_baseline_z_once_settled']:.2f}")
323
324 if drift_pick:
325     print("\ndrift vs attack (resource-footprint drift):")
326     print(f" best case {drift_pick['entity_id']}: {drift_pick['risk_before']:.3f} before -> "
327           f"{drift_pick['risk_during_shift']:.3f} during legitimate shift")
328     print(f" attacker {attack_pick} on its attack sessions: {attack_risk:.3f}")
329     print(f" across {population['n_drift_entities_considered']} drift entities, median risk "
330           f"{population['median_risk_before']:.3f} -> "
331           f"{population['median_risk_during_shift']:.3f} "
332           f"(rose for {population['n_entities_score_rose']})")

```

```
if __name__ == "__main__":  
    main()
```

tests/conftest.py

```
1 from datetime import timedelta
2
3 import pytest
4
5 from src.generator import config
6
7
8 @pytest.fixture
9 def small_config(monkeypatch):
10     monkeypatch.setattr(config, "N_USERS", 10)
11     monkeypatch.setattr(config, "N_SERVICE_ACCOUNTS", 3)
12     monkeypatch.setattr(config, "N_EDGE_DEVICES", 4)
13     monkeypatch.setattr(config, "SIM_DAYS", 12)
14     monkeypatch.setattr(config, "SIM_END", config.SIM_START + timedelta(days=12))
15     return config
```

tests/test_generator.py

```
1 import pandas as pd
2
3 from src.generator.config import ALL_LABELS
4 from src.generator.entities import build_entities
5 from src.generator.generate_dataset import COLUMN_ORDER, build_dataset
6
7
8 def test_build_entities_counts(small_config):
9     profiles = build_entities(seed=1)
10     n_users = sum(1 for p in profiles.values() if p["entity_type"] == "user")
11     n_svc = sum(1 for p in profiles.values() if p["entity_type"] == "service_account")
12     n_dev = sum(1 for p in profiles.values() if p["entity_type"] == "edge_device")
13     assert n_users == small_config.N_USERS
14     assert n_svc == small_config.N_SERVICE_ACCOUNTS
15     assert n_dev == small_config.N_EDGE_DEVICES
16
17
18 def test_build_dataset_schema_and_labels(small_config):
19     df, entities_df = build_dataset(seed=1)
20
21     for col in COLUMN_ORDER + ["label"]:
22         assert col in df.columns
23
24     assert set(df["label"].unique()) <= set(ALL_LABELS)
25     assert df["entity_id"].isin(entities_df["entity_id"]).all()
26     assert not df["timestamp"].isna().any()
27     assert (df["session_duration"] > 0).all()
28
29
30 def test_attack_rate_is_bounded(small_config):
31     df, _ = build_dataset(seed=2)
32     anomaly_rate = (df["label"] != "normal").mean()
33     assert 0 < anomaly_rate < 0.25
34
35
36 def test_low_and_slow_exfil_present_and_off_hours(small_config):
37     df, _ = build_dataset(seed=5)
```

```

38     exfil = df[df["label"] == "low_and_slow_exfil"]
39     assert len(exfil) > 0
40
41     hours = pd.to_datetime(exfil["timestamp"]).dt.hour
42     business_hours = hours.between(8, 17).mean()
43     assert business_hours < 0.15 # concentrated outside the normal working day
44
45
46 def test_low_and_slow_exfil_spans_multiple_days(small_config):
47     df, _ = build_dataset(seed=5)
48     exfil = df[df["label"] == "low_and_slow_exfil"].copy()
49     exfil["timestamp"] = pd.to_datetime(exfil["timestamp"])
50     span = exfil.groupby("entity_id")["timestamp"].agg(lambda s: (s.max() - s.min()).days)
51     # a genuine low-and-slow pattern builds up over multiple days, not a spike
52     assert (span >= 3).any()

```

tests/test_features.py

```

1  import pandas as pd
2
3  from src.features.build_features import FEATURE_COLUMNS, build_features
4  from src.generator.generate_dataset import COLUMN_ORDER, build_dataset
5
6
7  def _access_log(small_config):
8      df, _ = build_dataset(seed=3)
9      return df[COLUMN_ORDER].copy()
10
11
12 def test_no_nans_in_feature_columns(small_config):
13     features = build_features(_access_log(small_config))
14     assert features[FEATURE_COLUMNS].isna().sum().sum() == 0
15
16
17 def test_first_session_is_cold_start(small_config):
18     features = build_features(_access_log(small_config))
19     first_rows = features.sort_values(["entity_id", "timestamp"]).groupby("entity_id").first()
20     assert (first_rows["entity_history_length"] == 0).all()
21     assert (first_rows["is_cold_start"] == 1).all()
22     assert (first_rows["hours_since_last_session"] == 9999.0).all()
23
24
25 def test_geo_velocity_known_pair():
26     rows = pd.DataFrame([
27         {
28             "session_id": "s1", "entity_id": "e1", "entity_type": "user",
29             "timestamp": "2026-01-01 00:00:00", "source_ip": "1.1.1.1",
30             "geo_lat": 0.0, "geo_lon": 0.0, "geo_city": "a",
31             "resource_accessed": "r1", "auth_method": "password",
32             "auth_success": True, "session_duration": 100, "command_sequence": "",
33             "device_os": "os1", "device_mac": "AA",
34         },
35         {
36             "session_id": "s2", "entity_id": "e1", "entity_type": "user",
37             "timestamp": "2026-01-01 01:00:00", "source_ip": "1.1.1.1",
38             "geo_lat": 1.0, "geo_lon": 0.0, "geo_city": "b",
39             "resource_accessed": "r1", "auth_method": "password",

```

```

40         "auth_success": True, "session_duration": 100, "command_sequence": "",
41         "device_os": "os1", "device_mac": "AA",
42     },
43 ])
44 features = build_features(rows)
45 second = features[features["session_id"] == "s2"].iloc[0]
46 assert second["geo_distance_from_prev_km"] > 100
47 assert second["geo_velocity_kmh"] > 100
48 assert second["is_new_resource"] == 0
49 assert second["is_device_mismatch"] == 0
50
51
52 def test_resource_breadth_7d_covers_at_least_the_24h_window(small_config):
53     features = build_features(_access_log(small_config))
54     assert (
55         features["entity_resource_breadth_7d"] >= features["entity_resource_breadth_24h"]
56     ).all()
57
58
59 def test_resource_breadth_7d_sees_further_back_than_24h():
60     rows = pd.DataFrame([
61         {
62             "session_id": "s1", "entity_id": "e1", "entity_type": "user",
63             "timestamp": "2026-01-01 00:00:00", "source_ip": "1.1.1.1",
64             "geo_lat": 0.0, "geo_lon": 0.0, "geo_city": "a",
65             "resource_accessed": "r1", "auth_method": "password",
66             "auth_success": True, "session_duration": 100, "command_sequence": "",
67             "device_os": "os1", "device_mac": "AA",
68         },
69         {
70             "session_id": "s2", "entity_id": "e1", "entity_type": "user",
71             "timestamp": "2026-01-06 00:00:00", "source_ip": "1.1.1.1",
72             "geo_lat": 0.0, "geo_lon": 0.0, "geo_city": "a",
73             "resource_accessed": "r2", "auth_method": "password",
74             "auth_success": True, "session_duration": 100, "command_sequence": "",
75             "device_os": "os1", "device_mac": "AA",
76         },
77     ])
78     features = build_features(rows)
79     second = features[features["session_id"] == "s2"].iloc[0]
80     # r1 was 5 days ago: outside the 24h window, still inside the 7-day one
81     assert second["entity_resource_breadth_24h"] == 1
82     assert second["entity_resource_breadth_7d"] == 2
83
84
85 def test_device_mismatch_flagged():
86     rows = pd.DataFrame([
87         {
88             "session_id": "s1", "entity_id": "e1", "entity_type": "edge_device",
89             "timestamp": "2026-01-01 00:00:00", "source_ip": "1.1.1.1",
90             "geo_lat": 0.0, "geo_lon": 0.0, "geo_city": "a",
91             "resource_accessed": "r1", "auth_method": "certificate",
92             "auth_success": True, "session_duration": 10, "command_sequence": "",
93             "device_os": "EdgeOS-3.2", "device_mac": "AA:AA:AA:AA:AA:AA",
94         },
95         {
96             "session_id": "s2", "entity_id": "e1", "entity_type": "edge_device",

```

```

97         "timestamp": "2026-01-01 00:10:00", "source_ip": "1.1.1.1",
98         "geo_lat": 0.0, "geo_lon": 0.0, "geo_city": "a",
99         "resource_accessed": "r1", "auth_method": "certificate",
100        "auth_success": True, "session_duration": 10, "command_sequence": "",
101        "device_os": "RTOS-1.8", "device_mac": "BB:BB:BB:BB:BB:BB",
102    },
103    ])
104    features = build_features(rows)
105    second = features[features["session_id"] == "s2"].iloc[0]
106    assert second["is_device_mismatch"] == 1

```

tests/test_models.py

```

1  import numpy as np
2
3  from src.features.build_features import build_features
4  from src.generator.config import ALL_LABELS
5  from src.generator.generate_dataset import COLUMN_ORDER, build_dataset
6  from src.models.baseline import BaselineProfiler
7  from src.models.classifier import TabularClassifier
8  from src.models.cold_start import blend_cold_start_scores
9  from src.models.sequence_model import SequenceAnomalyModel, build_sequences
10
11
12  def _features_with_labels(small_config):
13      df, _ = build_dataset(seed=4)
14      access_log = df[COLUMN_ORDER].copy()
15      features = build_features(access_log)
16      return features.merge(df[["session_id", "label"]], on="session_id")
17
18
19  def test_baseline_profiler_scores_in_unit_range(small_config):
20      features = _features_with_labels(small_config)
21      profiler = BaselineProfiler().fit(features)
22      scores = profiler.score(features)
23      assert len(scores) == len(features)
24      assert scores.min() >= 0.0
25      assert scores.max() <= 1.0 + 1e-9
26
27
28  def test_cold_start_blend_shrinks_toward_prior(small_config):
29      features = _features_with_labels(small_config)
30      raw = np.random.RandomState(0).rand(len(features))
31      prior = {t: 0.5 for t in features["entity_type"].unique()}
32      blended = blend_cold_start_scores(features, raw, prior)
33      cold_mask = features["is_cold_start"] == 1
34      assert (blended[cold_mask.values] != raw[cold_mask.values]).any()
35
36
37  def test_tabular_classifier_predicts_known_labels(small_config):
38      features = _features_with_labels(small_config)
39      clf = TabularClassifier().fit(features)
40      preds = clf.predict(features)
41      assert set(preds) <= set(ALL_LABELS)
42      assert len(preds) == len(features)
43
44

```

```

45 def test_sequence_model_output_shape_and_normalization(small_config):
46     features = _features_with_labels(small_config)
47     x, y, session_ids = build_sequences(features, window=5)
48     assert x.shape[0] == len(features)
49     assert x.shape[1] == 5
50     assert len(session_ids) == len(features)
51
52     model = SequenceAnomalyModel(window=5, epochs=1).fit(x, y)
53     proba = model.predict_proba(x)
54     assert proba.shape == (len(features), len(ALL_LABELS))
55     np.testing.assert_allclose(proba.sum(axis=1), 1.0, atol=1e-4)

```

tests/test_cold_start.py

```

1  import numpy as np
2  import pandas as pd
3
4  from src.features.build_features import COLD_START_THRESHOLD
5  from src.models.cold_start import blend_cold_start_scores, cold_start_note
6
7
8  def _frame(history_lengths, entity_type="edge_device"):
9      return pd.DataFrame({
10         "entity_history_length": history_lengths,
11         "entity_type": [entity_type] * len(history_lengths),
12     })
13
14
15  def test_first_session_takes_the_population_prior():
16      frame = _frame([0])
17      blended = blend_cold_start_scores(frame, np.array([0.9]), {"edge_device": 0.2})
18      assert blended[0] == 0.2
19
20
21  def test_settled_entity_keeps_its_own_score():
22      frame = _frame([COLD_START_THRESHOLD, COLD_START_THRESHOLD + 50])
23      raw = np.array([0.9, 0.9])
24      blended = blend_cold_start_scores(frame, raw, {"edge_device": 0.2})
25      np.testing.assert_allclose(blended, raw)
26
27
28  def test_blend_moves_monotonically_toward_own_score():
29      history = np.arange(0, COLD_START_THRESHOLD + 1)
30      frame = _frame(history)
31      raw = np.full(len(history), 0.9)
32      blended = blend_cold_start_scores(frame, raw, {"edge_device": 0.1})
33
34      assert np.all(np.diff(blended) > 0)
35      assert blended[0] == 0.1
36      assert blended[-1] == 0.9
37
38
39  def test_blend_suppresses_a_spurious_new_entity_spike():
40      """A brand-new device looks anomalous on every novelty feature; the blend
41      is what stops that from reaching the alert queue."""
42      frame = _frame([0, 1])
43      blended = blend_cold_start_scores(frame, np.array([0.95, 0.95]), {"edge_device": 0.15})

```

```

44     assert (blended < 0.5).all()
45
46
47 def test_unknown_entity_type_falls_back_to_mean():
48     frame = _frame([0], entity_type="new_kind_of_thing")
49     raw = np.array([0.8, 0.4])[:1]
50     blended = blend_cold_start_scores(frame, raw, {"edge_device": 0.2})
51     assert np.isfinite(blended).all()
52
53
54 def test_cold_start_note_only_below_threshold():
55     assert cold_start_note(0) is not None
56     assert cold_start_note(COLD_START_THRESHOLD - 1) is not None
57     assert cold_start_note(COLD_START_THRESHOLD) is None

```

tests/test_feedback.py

```

1  from pathlib import Path
2
3  import numpy as np
4  import pandas as pd
5
6  from src.dashboard import feedback
7
8
9  def _db(tmp_path):
10     return tmp_path / "feedback.db"
11
12
13 def _queue(n=10_000):
14     """A score distribution shaped like the real one: the top 1% packed into a
15     narrow score band just above the cutoff, which is exactly the clustering
16     that made the fixed score cap ineffective.
17
18     Alerts run low->high, split into named blocks by queue position:
19     ``marginal`` bottom of the queue, sitting on the cutoff
20     ``midblock`` middle of the queue - score margin < 0.10 but percentile
21     margin > the percentile cap, so the two schemes disagree
22     ``topblock`` top of the queue, far enough above the cutoff that even the
23     old score cap could not remove it (the dev_0025 case)
24     """
25     n_alerts = n // 100
26     rng = np.random.default_rng(0)
27     scores = np.concatenate([
28         rng.uniform(0.0, 0.885, n - n_alerts),
29         np.linspace(0.886, 1.0, n_alerts),      # queue: 0.114 wide, like the real one
30     ])
31
32     entity = [f"bulk{i % 50}" for i in range(n - n_alerts)]
33     for i in range(n_alerts):
34         frac = i / (n_alerts - 1)
35         if frac < 0.10:
36             entity.append("marginal")
37         elif 0.45 <= frac < 0.55:
38             entity.append("midblock")
39         elif frac >= 0.90:
40             entity.append("topblock")

```

```

41         else:
42             entity.append(f"filler{i}")
43
44     return pd.DataFrame({
45         "session_id": [f"s{i}" for i in range(n)],
46         "entity_id": entity,
47         "risk_score": scores,
48     })
49
50
51 def test_no_decisions_means_no_adjustment(tmp_path):
52     decisions = feedback.load_decisions(_db(tmp_path))
53     assert decisions.empty
54     assert feedback.entity_adjustments(decisions) == {}
55
56
57 def test_decisions_round_trip(tmp_path):
58     db = _db(tmp_path)
59     feedback.record_decision("s1", "user_1", feedback.DISMISS, db)
60     feedback.record_decision("s2", "user_1", feedback.CONFIRM, db)
61
62     decisions = feedback.load_decisions(db)
63     assert len(decisions) == 2
64     assert set(decisions["session_id"]) == {"s1", "s2"}
65
66
67 def test_revisiting_a_session_overwrites_not_duplicates(tmp_path):
68     db = _db(tmp_path)
69     feedback.record_decision("s1", "user_1", feedback.DISMISS, db)
70     feedback.record_decision("s1", "user_1", feedback.CONFIRM, db)
71
72     decisions = feedback.load_decisions(db)
73     assert len(decisions) == 1
74     assert decisions.iloc[0]["decision"] == feedback.CONFIRM
75
76
77 def test_dismissals_lower_and_confirmations_raise(tmp_path):
78     db = _db(tmp_path)
79     for i in range(3):
80         feedback.record_decision(f"d{i}", "noisy_entity", feedback.DISMISS, db)
81     feedback.record_decision("c0", "real_threat", feedback.CONFIRM, db)
82
83     adjustments = feedback.entity_adjustments(feedback.load_decisions(db))
84     assert adjustments["noisy_entity"] == -3 * feedback.DISMISS_STEP
85     assert adjustments["real_threat"] == feedback.CONFIRM_STEP
86
87
88 def test_adjustment_is_capped(tmp_path):
89     db = _db(tmp_path)
90     for i in range(feedback.MAX_STEPS + 10):
91         feedback.record_decision(f"d{i}", "very_noisy", feedback.DISMISS, db)
92
93     adjustments = feedback.entity_adjustments(feedback.load_decisions(db))
94     assert adjustments["very_noisy"] == -feedback.MAX_STEPS * feedback.DISMISS_STEP
95
96
97 def test_mixed_decisions_net_out(tmp_path):

```

```

98     db = _db(tmp_path)
99     feedback.record_decision("d0", "mixed", feedback.DISMISS, db)
100    feedback.record_decision("d1", "mixed", feedback.DISMISS, db)
101    feedback.record_decision("c0", "mixed", feedback.CONFIRM, db)
102
103    adjustments = feedback.entity_adjustments(feedback.load_decisions(db))
104    assert adjustments["mixed"] == feedback.CONFIRM_STEP - 2 * feedback.DISMISS_STEP
105
106
107    def test_apply_adjustments_reranks_and_stays_in_range():
108        scored = pd.DataFrame({
109            "session_id": ["s1", "s2", "s3"],
110            "entity_id": ["noisy", "clean", "noisy"],
111            "risk_score": [0.90, 0.88, 0.02],
112        })
113        adjusted = feedback.apply_adjustments(scored, {"noisy": -0.10})
114
115        assert adjusted.loc[0, "effective_risk"] == 0.80
116        assert adjusted.loc[1, "effective_risk"] == 0.88
117        # the clean entity now outranks the dismissed one
118        assert adjusted.sort_values("effective_risk", ascending=False).iloc[0]["entity_id"] == "clean"
119        # and nothing escapes [0, 1]
120        assert adjusted["effective_risk"].between(0, 1).all()
121
122
123    def test_unknown_decision_rejected(tmp_path):
124        try:
125            feedback.record_decision("s1", "e1", "maybe", _db(tmp_path))
126        except ValueError:
127            return
128        raise AssertionError("expected ValueError for an unknown decision")
129
130
131    # --- percentile-rank capping -----
132
133
134    def test_percentile_ranks_span_the_distribution():
135        ranks = feedback.percentile_ranks([0.1, 0.5, 0.9, 0.3])
136        assert ranks.min() == 0.0
137        assert ranks.max() == 100.0
138        # order is preserved
139        assert list(np.argsort(ranks)) == list(np.argsort([0.1, 0.5, 0.9, 0.3]))
140
141
142    def test_zero_movement_round_trips_exactly():
143        """Rank -> score has to be an exact identity, or every untouched entity
144        drifts slightly the moment anyone triages anything."""
145        scores = np.linspace(0.0, 1.0, 500) ** 2
146        ranks = feedback.percentile_ranks(scores)
147        np.testing.assert_allclose(np.quantile(scores, ranks / 100.0), scores)
148
149
150    def test_net_steps_are_uncapped():
151        """The bound belongs on queue movement, not on how often an analyst clicks."""
152        decisions = pd.DataFrame({
153            "session_id": [f"s{i}" for i in range(20)],
154            "entity_id": ["noisy"] * 20,

```

```

155         "decision": [feedback.DISMISS] * 20,
156         "decided_at": ["2026-01-01"] * 20,
157     })
158     assert feedback.entity_net_steps(decisions) == {"noisy": -20}
159
160
161 def _still_queued(frame, entity, cutoff):
162     rows = frame[(frame["entity_id"] == entity) & (frame["risk_score"] >= cutoff)]
163     return int((rows["effective_risk"] >= cutoff).sum())
164
165
166 def test_percentile_movement_is_capped_regardless_of_decision_count():
167     scored = _queue()
168     modest = feedback.apply_percentile_adjustments(scored, {"midblock": -5})
169     extreme = feedback.apply_percentile_adjustments(scored, {"midblock": -500})
170
171     for frame in (modest, extreme):
172         moved = frame[frame["entity_id"] == "midblock"]["pct_movement"]
173         assert moved.abs().max() <= feedback.PERCENTILE_CAP + 1e-9
174
175     # 5 decisions already reaches the cap, so 500 buys nothing extra
176     np.testing.assert_allclose(
177         modest[modest["entity_id"] == "midblock"]["effective_risk"].values,
178         extreme[extreme["entity_id"] == "midblock"]["effective_risk"].values,
179     )
180
181
182 def test_percentile_cap_protects_a_mid_queue_entity_the_score_cap_does_not():
183     """The case the change exists for: a mid-queue entity whose score margin is
184     under 0.10 gets cleared out entirely by the old cap, and stays visible under
185     the percentile cap because it is nowhere near the bottom of the queue."""
186     scored = _queue()
187     cutoff = scored["risk_score"].quantile(0.99)
188     n_alerts = int(((scored["entity_id"] == "midblock")
189                     & (scored["risk_score"] >= cutoff)).sum())
189     assert n_alerts > 0
190
191     old = feedback.apply_adjustments(scored, {"midblock": -5 * feedback.DISMISS_STEP})
192     new = feedback.apply_percentile_adjustments(scored, {"midblock": -5})
193
194     assert _still_queued(old, "midblock", cutoff) == 0
195     assert _still_queued(new, "midblock", cutoff) == n_alerts
196
197
198
199 def test_top_of_queue_entity_is_bounded_under_both_schemes():
200     """Honest counterpart: an entity far enough above the cutoff was already
201     safe under the old cap, so the change buys nothing for it."""
202     scored = _queue()
203     cutoff = scored["risk_score"].quantile(0.99)
204     n_alerts = int(((scored["entity_id"] == "topblock")
205                     & (scored["risk_score"] >= cutoff)).sum())
206
207     old = feedback.apply_adjustments(scored, {"topblock": -5 * feedback.DISMISS_STEP})
208     new = feedback.apply_percentile_adjustments(scored, {"topblock": -5})
209
210     assert _still_queued(old, "topblock", cutoff) == n_alerts
211     assert _still_queued(new, "topblock", cutoff) == n_alerts

```

```

212
213
214 def test_percentile_cap_still_lets_marginal_alerts_be_dismissed_away():
215     """The flip side: an entity sitting on the cutoff is still removable, so
216 legitimate triage of borderline noise keeps working."""
217     scored = _queue()
218     cutoff = scored["risk_score"].quantile(0.99)
219     new = feedback.apply_percentile_adjustments(scored, {"marginal": -5})
220
221     rows = new[(new["entity_id"] == "marginal") & (new["risk_score"] >= cutoff)]
222     assert len(rows) > 0
223     assert (rows["effective_risk"] < cutoff).any()
224
225
226 def test_percentile_confirmations_raise_and_dismissals_lower():
227     scored = _queue()
228     adjusted = feedback.apply_percentile_adjustments(
229         scored, {"midblock": -5, "marginal": 5}
230     )
231
232     assert (adjusted[adjusted["entity_id"] == "midblock"]["pct_movement"] <= 0).all()
233     assert (adjusted[adjusted["entity_id"] == "marginal"]["pct_movement"] >= 0).all()
234     # untouched entities do not move at all
235     assert (adjusted[adjusted["entity_id"] == "bulk0"]["pct_movement"] == 0).all()
236
237
238 def test_percentile_edges_clip_at_the_distribution_boundary():
239     """Entities already at the top or bottom cannot be pushed off the scale."""
240     scored = _queue()
241     top_entity = scored.loc[scored["risk_score"].idxmax(), "entity_id"]
242     bottom_entity = scored.loc[scored["risk_score"].idxmin(), "entity_id"]
243
244     adjusted = feedback.apply_percentile_adjustments(
245         scored, {top_entity: 500, bottom_entity: -500}
246     )
247     assert adjusted["effective_pct_rank"].between(0, 100).all()
248     assert adjusted["effective_risk"].between(0, 1).all()
249
250
251 def test_apply_feedback_dispatches_and_rejects_unknown_mode():
252     scored = _queue()
253     decisions = pd.DataFrame({
254         "session_id": ["s0"], "entity_id": ["e0"],
255         "decision": [feedback.DISMISS], "decided_at": ["2026-01-01"],
256     })
257
258     for mode in (feedback.CAP_MODE_SCORE, feedback.CAP_MODE_PERCENTILE):
259         out = feedback.apply_feedback(scored, decisions, mode=mode)
260         assert {"risk_adjustment", "effective_risk"} <= set(out.columns)
261
262     try:
263         feedback.apply_feedback(scored, decisions, mode="sideways")
264     except ValueError:
265         return
266     raise AssertionError("expected ValueError for an unknown cap mode")
267
268

```

